

Learning Cluttered Scene Manipulation with Goals

Lirui Wang

A senior thesis submitted in partial fulfillment of the requirements for the degree of Bachelor of
Science With Departmental Honors

Paul G. Allen School of Computer Science and Engineering
University of Washington

June 2020

Supervisor:

Professor Dieter Fox

Approved by _____

Presentation of work given on _____

©2020 Lirui Wang
UW Computer Science and Engineering

©Copyright 2020

Lirui Wang

To everyone I encountered in the journey, with great appreciation.

ACKNOWLEDGMENTS

I've been really lucky to work on robotics research at the UW CSE Department. I want to especially thank Professor Fox for giving me a chance to work on robotics far before I was qualified to do so. His passion and experience have inspired me to explore many interesting ideas and problems such as pose estimation and grasping. Many others have also mentored me. I would like to thank Dr. Yu Xiang for the guidance over the years. His support is encouraging – especially the time and effort taken to meet with me one-on-one and the responsive online communication. Besides technical ideas from 3D transform basics to the network design, he has shown me an ability to analyze the essence of research questions and instilled a desire to build systems that actually work. I would also like to thank Xiangyun for his patience – I completely destroyed his productivity by constantly consulting compile error and buggy code, and pouring very personal thoughts on my own research. Sharing the space for around three years, I'm also grateful to all members in the RSE Lab. Finally, I wish to thank my family and friends for their support and encouragement throughout my study. I hope I can repay their love and kindness in the years to come.

TABLE OF CONTENTS

	Page
List of Figures	vii
List of Tables	ix
Chapter 1: Introduction	1
1.1 Deep Learning for Object Pose Refinement and Tracking	1
1.2 Optimization and Learning for Manipulation Planning	2
1.3 Imitation and Reinforcement Learning for End-to-End Goal Conditioned Policy . .	2
Chapter 2: Deep Learning for Pose Refinement and Tracking	4
2.1 Introduction	4
2.2 Preliminaries and Related Works	5
2.3 Methodology	7
2.4 Experiments	15
2.5 Conclusion	16
2.6 Note	16
Chapter 3: Optimization and Learning for Manipulation Planning	18
3.1 Introduction	18
3.2 Preliminaries and Related Works	21
3.3 Methodology	25
3.4 Experiments	33
3.5 Conclusion	40
3.6 Note	41
Chapter 4: Imitation and Reinforcement Learning for Goal Conditioned Grasping Policy	42
4.1 Introduction	42
4.2 Preliminaries and Related Works	44
4.3 Methodology	50
4.4 Experiments	54

4.5 Conclusion	57
4.6 Note	58
Chapter 5: Afterword	59
Bibliography	63

LIST OF FIGURES

Figure Number		Page
1.1	We consider a standard robotic system composed of perception, planning, and control, inside a feedback loop. We propose that in each component, goal information (included in the red block) can be used to provide guidance.	1
2.1	In perception system, the task is to estimate pose that explains the observation. Treating matching with observed image as goal, DeepIM [70] iteratively refines the pose estimate and by comparing against environment observation.	5
2.2	DeepIM Network Architecture [70]	9
2.4	Pose Distribution Visualization	12
2.6	Generated Occlusion Images. We can observe that the PNP occlusion look more natural than Naive occlusion.	13
2.8	Refinement Performance on YCB Video Dataset.	14
2.10	Robust Refinement Performance in the Real World.	14
2.12	DeepIM Application on Robot Pose Tracking and Augmented Reality.	15
2.14	YCB Objects Projection Performance Plot. <i>deepim</i> and <i>posecnn</i> refer to keyframe estimation in YCB-Video Dataset. We observe an expected improvement through iterative refinements. The rest are tracking for each frame in YCB-Video. We observe a stable performance of pose tracking using DeepIM.	17
3.1	Our planning system integrate grasp and motion planning into a single optimization frameworks to output a trajectory for execution. Our method proposes an online goal selection and refinement mechanism, to leverage the goal set information about the target object.	19
3.2	A planning scene example. The goal set, initial grasp and selected grasp are yellow, red and green, respectively. Our algorithm synthesizes and selects grasps online and generates motion trajectories to reach the selected grasp.	20
3.3	Our Optimization-based Motion and Grasp Planner (OMG Planner) consists of three components. The motion planner component optimizes the current trajectory denoted by the red line for a fixed target goal. The online learning component considers the goal set (yellow), and update the initial grasp (red) with a new grasp (green). The grasp refinement component iteratively improves the grasp (green to yellow). The refined grasp is used as the target goal for the next iteration.	25

3.4	Online goal selection algorithm at the i th iteration. The goal costs $c(g)$ are denoted inside red circles. In this example, the online learning algorithm is about to shift goal from A to B , which yields a collision-free yet shorter trajectory.	29
3.5	Red points denote the sampled contact points u_h with normals u_h^n and purple points denote the corresponding object points u_o with normals u_o^n . The correspondence lines are colored in cyan. Body points u range from blue to green, denoting from far to close with respect to objects.	31
3.6	Three initial grasp sets of the mustard bottle.	34
3.7	Grasp optimization results. The intermediate grasps are colored from green to red and yellow.	38
3.8	Planning scene examples. Red trajectories come from CHOMP, and green trajectories come from our OMG Planner.	38
3.9	Left) Planning scene examples in Pybullet. Right) Successful real world grasping examples by executing trajectories from our planner using estimated 6D object poses from [22]	38
3.11	Ablation study of n and λ . The metrics (except planning) are scaled between 0 and 1. Left) We see a decrease of smoothness and a increase of success rate due to finer representation. The planning time drops first due to more solvable problems and then increases due to increase of computation. Right) The smoothness weight λ around 0.1 acts as a good balance in terms of collision and planning success.	40
4.1	Different from traditional sense, plan, act paradigm, an end-to-end goal conditioned policy for control uses goal as a proxy to circumvent long-horizon planning.	43
4.2	Our Goal Conditioned Grasping policy framework. Our grasp framework (sampler h and evaluator e), based on [80], takes in the observed point cloud x and synthesize a goal grasp for the policy π . The red gripper on the top denotes current gripper pose, yellow denotes sample grasps, and green denotes the selected grasp after refinement.	44
4.3	Actor-Critic Method train value and policy functions simultaneously.	48
4.5	Green denotes the grasp with highest value and red denotes other sampled candidate grasps, which shows that the finetuned value function remains picking high quality grasp.	52
4.6	The figures show two grasping episodes to compare reaching good grasp to reaching a bad grasp. Red denotes a bad sampled goal grasp and green denotes the grasp selected by planner [129].	53
4.7	The figures show examples of successful grasping episodes from our learned policy in the simulator [20].	57

LIST OF TABLES

Table Number	Page
2.1 Comparison between Different Training Pose Distribution	12
2.2 DeepIM Ablation Study Performance. <i>PoseCNN</i> [132] refers to initialize pose estimate with PoseCNN. <i>Reinitialize</i> refers to single frame pose refinement by DeepIM. <i>Tracking</i> refers to initialize pose estimate with the refined pose from DeepIM.	16
3.1 Comparison between two baselines and four online learning algorithms for grasp selection.	36
3.2 Grasp costs for six YCB objects before and after refinement. “S”, “G” and “A” denotes the simulated set, the graspt set, and the approach set, respectively. Planning and execution success are from 10 runs of the 100 planning scenes.	37
3.3 Comparison on planning methods for joint motion and grasp planning. CHOMP [98], CHOMP-C [26], FMT [50], RRT-Connect [60].	39
4.1 Comparison for our imitation learning algorithms. Our goal conditioned framework performs more stably for all objects. The addition of trajectory state loss has a comparable performance.	56
4.2 We note that the experiments on this table use goals from the expert, and therefore the performance is slightly better than Table 4.1. The table and empirical observation show that RL improves the performance over difficult scenes.	57

Paul G. Allen School of Computer Science and Engineering
University of Washington

Abstract

Learning Cluttered Scene Manipulation with Goals

Lirui Wang

Supervisory Committee:

Building systems through strong modularization remains the predominant paradigm for high DoF manipulation systems. While most industrial robots are carefully tuned and limited to work in highly structured environments, the recent machine learning crush pushes researchers to propose more data-driven solutions from both inference and decision-making aspects. However, many such systems often end up solving an ill-posed or overly strict problem with limited robustness and interpretation. This thesis seeks to analyze how a robot system can make use of goal information in perception, planning, and control. This helps understand the limitations and potentials of components in a manipulation pipeline, a crucial step towards their principled enhancement for an end-to-end system.

Perception allows robots to reason about the surrounding environment. Treating matching observed appearance of the target object as the goal for a pose estimation problem, we investigated training Deep Iterative Matching Network for pose refinement on synthetic data. We also experiment with pose tracking and other real-world applications of the framework.

Planning allows robots to find feasible trajectories to manipulate target objects. While grasp and motion planning are often tackled separately, we have delved into an integration of these two planning problems. We proposed our optimization-based planner that applies online learning to select goal configurations for grasping and introduce a grasp refinement procedure.

Control allows robots to operate safely and precisely in cluttered scenes. We studied a goal-conditioned visuomotor policy based on point cloud observation that can enable efficient generalization in the real world. We further combined reinforcement learning with imitation learning to achieve goal synthesis, selection, and reaching in an end-to-end fashion.

Chapter 1

INTRODUCTION

In the typical sense-plan-act paradigm, perception provides a model of the environment, in which a motion planner finds an optimal, collision free path that is then tracked by a stiff and accurate controller. Considering a standard manipulation task where a robot aims to pick and place a target object on a table without colliding with other objects. While typical industry applications require heavy engineering, this problem remains challenging in unstructured environments. To help with the drawbacks such as lack of interpretability and robustness of a general end-to-end framework, we investigated the role of goal information for perception, planning, and control from a learning perspective, shown in Fig 1.1. To achieve successful robot learning, a proposed method should make good use of domain knowledge, data, and learning algorithm. Moreover, one should leverage expressive function approximators and efficient algorithms to enable robots to continually learn from interacting with and observing the world.

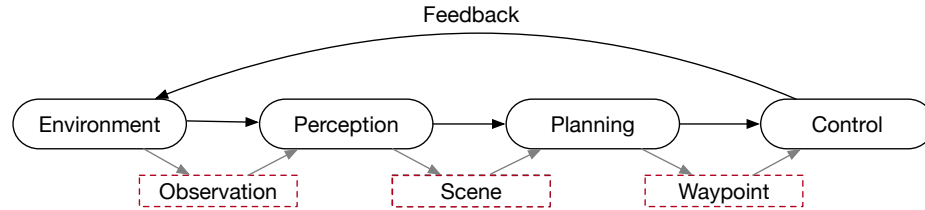


Figure 1.1: We consider a standard robotic system composed of perception, planning, and control, inside a feedback loop. We propose that in each component, goal information (included in the red block) can be used to provide guidance.

1.1 Deep Learning for Object Pose Refinement and Tracking

The pose of an object is generally defined as a $\mathbb{SE}(3)$ transform (translation and rotation) of an interested object in camera frame. Pose estimation has been studied extensively by computer vision and

robotics researchers for its importance in various applications such as robot manipulation and virtual reality. While direct pose regression from RGB image is limited by various factors, DeepIM (Deep Iterative Matching) [70] takes an iterative approach to refine initial pose estimation by matching rendered image against real image, as the observed goal for perception task. The network is trained to iteratively predict a relative pose transformation using an untangled representation of 3D location and 3D orientation. In our following work, we address the performance of DeepIM on YCB dataset with synthetic training and the performance of DeepIM on tracking tasks and real-world applications.

1.2 Optimization and Learning for Manipulation Planning

Optimization and machine learning are two mathematics fields that have many applications in robotics. In robot manipulation, planning the motion of a robot manipulator to grasp an object is a fundamental problem. A manipulation planner needs to generate a trajectory of the manipulator to avoid obstacles in the environment and plan an end-effector pose for grasping. While trajectory planning and grasp planning are often tackled separately, how to efficiently integrate the two planning problems remains a challenge. Therefore, we investigate a novel method for joint motion and grasp planning. Our Optimization-based Motion and Grasp planner (OMG) integrates manipulation trajectory optimization with online grasp synthesis and selection, where we apply online learning techniques to select goal configurations for grasping, and introduce a new grasp synthesis algorithm to generate grasps online. We evaluate our planning approach and demonstrate that our method generates robust and efficient motion plans for grasping objects in cluttered scenes.

1.3 Imitation and Reinforcement Learning for End-to-End Goal Conditioned Policy

Imitation learning (IL) is the study of algorithms that attempt to improve performance by mimicking a teachers decisions and behaviors. Reinforcement learning (RL) is a technique concerned with how agents ought to take actions in an environment in order to maximize the notion of cumulative reward, often in a Markov Decision Process (MDP) setting. Different from classical machine learning inference problem that uses a fixed batch of data, these techniques can interact with the environment to improve the decision-making ability of an agent. For grasping with 6 degrees of

freedoms, a learner needs to have an awareness of the shape geometry and long-horizon reasoning ability. We therefore use goal information to help learning efficiency by explicitly modeling a goal selection and refinement mechanism. Preliminary experiments show that our goal-conditioned agent performs better compared to the baselines.

Chapter 2

DEEP LEARNING FOR POSE REFINEMENT AND TRACKING

2.1 Introduction

Localizing objects from cameras has a wide range of applications such as robot manipulation and virtual reality. The problem is challenging due to the variety of objects shapes, observed appearance, and environment factors such as lighting conditions and object occlusions. Various methods including depth-based, RGB-based and RGBD-based methods have been proposed to solve the problem. While depth cameras can be used for object pose estimation, such cameras have limitations with respect to frame rate, field of view, resolution, and depth range. Traditional computer vision pipelines for pose estimation starts with heavy feature engineering to estimate key points of the object [103] for template matching. After matching feature points, they often rely on efficient solver for matching object poses [65]. However, these methods require that there are rich textures on the objects as feature points for matching. As a result, they often fail short of handling texture-less objects. RGBD methods can improve the performance with depth data but their performance can suffer from occlusions. Alternatively methods that establish the 2D-3D correspondences for 6D pose estimation cannot handle symmetric object. More recent machine learning techniques often formulate the 6D pose estimation problem into a classification problem [48] by discretizing the pose space or regression problem [132]. However, those methods are not robust to small errors and not generalized well to unseen objects. A common way to improve the pose accuracy is through pose refinement: Given an initial pose estimation, a synthetic RGB image can be rendered and used to match against the target observed image. Then the pose can be iteratively improved to increase the matching score.

Deep Iterative Matching network (DeepIM) [70] is introduced with an untangled representation of the SE3 transformation between object poses to achieve accurate pose refinements. Shown in Fig 2.1, this method treats observed image as the goal, iteratively composing relative transform to match the goal image. This work conducts experiments extensively on YCB-Video dataset [132] and proposes several improvements and applications. First, we show that diverse training pose

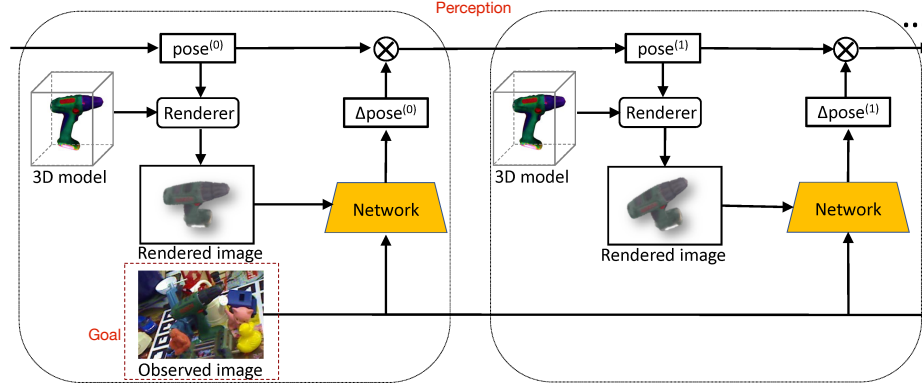


Figure 2.1: In perception system, the task is to estimate pose that explains the observation. Treating matching with observed image as goal, DeepIM [70] iteratively refines the pose estimate and by comparing against environment observation.

distribution and more realistic occlusion generation can help network performance on real occluded scene. Moreover, we extend the work to track object poses and achieved robust results under heavy occlusion. Finally, we also apply DeepIM on tracking robot end effector state and augmented reality applications.

2.2 Preliminaries and Related Works

2.2.1 Pose Estimation

Given an input image, the task of 6D object pose estimation is to infer the rigid transformation from the object coordinate system O to the camera coordinate system C . We assume that the 3D model of the object is available and the object coordinate system is defined in the 3D space of the model. The rigid transformation here consists of an $\mathbb{SE}(3)$ transform containing a 3D rotation R and a 3D translation T , where R can be represented in axis-angle representation, rotation matrix representation, euler angle representation and quaternion representation and T is the coordinate of the origin of O in the camera coordinate system C .

2.2.2 *Depth Based Method*

When depth information is available, it can be combined with RGB images to improve 6D pose estimation. With known camera intrinsics, a common strategy is to convert a depth image into a 3D point cloud, and then match the 3D model of an object against the observed point cloud. Since in general monocular pose estimation can be ill-posed depth information provides important information for translation estimation. This approach is widely used in many robotic applications. For pose refinement and tracking, the Iterative Closest Point (ICP) and its variants [15, 8] and Dense Articulated Real-Time Tracking (DART) [105] can align points efficiently in an optimization procedure thanks to the accelerated computations from GPU. Global registration methods compute pose alignments without an initial estimate. These methods are often expensive since they require global sampling and require computations of a fine-grained sampled correspondence. These methods are often sensitive to the initial estimate and may converge to local minima and their usage of texture information have little semantic meaning.

2.2.3 *RGB Based Method*

Without depth information, the recent development of convolutional neural networks (CNN) offers a potential for an end-to-end system to estimate and refine pose. A convolutional neural network is a series of convolution and pooling layers. It acts as a feature extractor in traditional computer vision technique and is often followed by a multi-layer perceptron or fully connected layers to regress the object pose. The convolutions are used to create representations of the input, and the pooling is used to downsize the data. CNN has been shown to have the potential to extract high-level semantic feature representations from images with proper training. CNN has been applied successfully to segmentation and detection tasks [100, 46] as the first steps for pose estimation. Its application in 3D vision typically requires more thoughtful design. A naive end-to-end framework that regresses object poses from image often fall shorts of estimation accuracy, since small errors in the classification or regression stage directly lead to pose mismatches.

The state-of-the-art methods [57, 94] augment deep learning based object detection or segmentation methods for 6D pose estimation. For instance, [94] utilize deep neural networks to detect 3D bounding box, and then compute the 6D pose by solving the PnP problem. Recent works such as

[115] use autoencoder to discretize and encode object poses. [22] further propose life-long pose learning in a self-supervised manner. At inference time, they map the object in the image to a vector and search for the most similar vector in a pre-generated codebook for pose estimation. [21] formulates the 6D object pose tracking problem in the Rao-Blackwellized particle filtering framework. Compared with traditional methods, deep learning based methods leverage powerful feature representation for pose estimation and thus achieve better performance.

2.2.4 *RGB-D Based Method*

When both RGB images and depth images are available, they can be combined to improve 6D pose estimation. Among many others, [132] has a refine stage using a customized ICP. [128] fuses depth information and color information for unified pose estimation and refinement. These methods typically make more explicit usage of 3D information provided by the depth image.

2.2.5 *Structured Pose Estimation and Refinement*

Two closely related works are [132, 70]. Their success as unified approaches for pose estimation and refinement has benefited from several ideas. In the imaging process, T determines the object location and scale in the image, while R affects the image appearance of the object according to the 3D shape and texture of the object. Since these two components of a rigid transform have distinct visual properties, it's important to design a convolutional neural network architecture that internally decouples the estimation of R and T . Moreover, a loss based on point distance in 3D gives rise to balanced regression of translation and rotation. Lastly, a reduction of 3D pose estimation to image plane plays a key role in efficient and generalized learning objective for neural networks. This chapter starts by introducing DeepIM and analyzing its design and structure. Then we propose a few modifications and improvements to the framework. At the end, we present experiment results that qualitatively and qualitatively demonstrate the performance of our extension.

2.3 *Methodology*

In this section, we investigate and analyze the framework of DeepIM and propose our extensions and applications.

2.3.1 DeepIM

DeepIM[70] predicts a relative SE3 transformation to match the observed and rendered image of an object in an iterative manner 2.2. By iteratively re-rendering the object based on the improved pose estimates, the two input images to the network become more and more similar, thereby enabling the network to generate more and more accurate pose estimate.

Zoom-In Operation

To obtain concrete and focused details from RGB image, a zoom-in operation[70] for rendered and observed image is performed before feeding them into the network. Specifically this preprocessing step zooms in an image patch centered at target object and performs bilinear up-sampling to achieve the same size as the original image while keep the aspect ratio of the object unchanged. This spatial transformation provides the network with more signals and information about the center of the object. Compared with attention mechanism that requires the network to implicitly learn the region of interests in the image, this method explicitly allows the network to focus on the relative transform.

Network Structure

DeepIM uses a FlowNet backbone [24]. Optical flow estimation provides a natural basis to estimate the appearance difference between two frames, similar to traditional template matching techniques in computer vision. Specifically, in the i th stage of the iterative matching, given a 6D pose estimate $p^{(i-1)}$ from the previous step, a synthetic image using the 3D object model is rendered with pose $p^{(i-1)}$. Typically 4 iterations are performed and one can observe a consistent pose improvement.

Untangled Transformation Representation

This representation [70] chooses the coordinate frames such that 3D object translations correspond to pixel-wise translations in the image plane along with a change in scale, and 3D rotations correspond to rotations around the image center along with a pan and tilt of the image. To estimate the relative translation, let $t_{tgt} = (x_{tgt}, y_{tgt}, z_{tgt})$ and $t_{src} = (x_{src}, y_{src}, z_{src})$ be the target translation and the source translation. The network is trained to regress to the relative translation $t_\delta = (v_x, v_y, v_z)$,

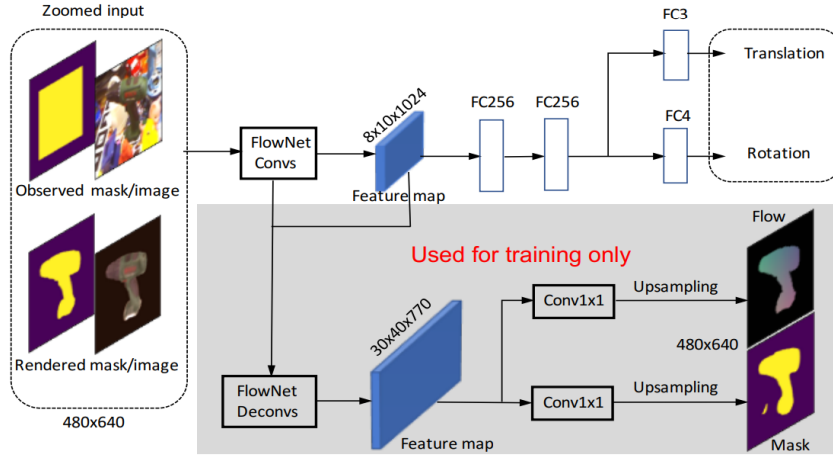


Figure 2.2: DeepIM Network Architecture [70]

where v_x and v_y denote the number of pixels the object should move along the image x-axis and y-axis and v_z is the scale change of the object:

$$v_x = f_x(x_{tgt}/z_{tgt} - x_{src}/z_{src})$$

$$v_y = f_y(y_{tgt}/z_{tgt} - y_{src}/z_{src})$$

$$v_z = \log(z_{tgt}/z_{src})$$

where f_x and f_y denote the focal lengths of the camera. For rotation, the center of rotation is moved from the origin of the camera to the center of the object in the camera frame, given by the current pose estimate. This estimate also specifies the center for the zoom-in operation and plays a key role in placing the real target in an approximate image patch. Moreover, it decomposes translation and rotation: a rotation would not change the translation of the object in the camera frame. It also removes the need for network to memorize object coordinates and allows generalization to unseen objects.

Point Matching Loss

Point Matching Loss is used in network training: Given the ground truth pose $p = [R|t]$ and the estimated pose $\hat{p} = [\hat{R}|\hat{t}]$, the point matching loss is computed as

$$L_{pose}(p, \hat{p}) = \frac{1}{n} \sum_{j=1}^n \left\| (Rx_j + t) - (\hat{R}x_j + \hat{t}) \right\|_1 \quad (2.1)$$

where x_j denotes a randomly selected 3D point on the object model and n is the total number of sample points used in computing the loss. This loss integrates the pose error as a whole and eliminates the issues to balance the translation and rotation errors. For symmetric objects, it becomes 3D Chamfer distance and takes closest points as target. Additional flow and mask losses are used to improve stability during training.

Training and Testing

In training time, 3D object models and images annotated with ground truth 6D object poses are required. By adding noises to the ground truth poses as the initial poses, the required observed and rendered inputs are generated for the network along with the pose target output, which is the pose difference between the ground truth pose and the noisy pose. Then the network is trained to predict the relative transformation between the initial pose and the target pose. Multiple iterations are performed during training as well to produce a similar training-test distribution match. Specifically, for each training image and pose, transformation predicted from the network is applied and use the transformed pose estimate as another training example for the network in the next iteration.

2.3.2 Extension and Application

Overall we make the following contributions to the original DeepIM. (1) We reproduce the DeepIM results on LINEMOD dataset. (2) We experiment with training DeepIM on synthetic data only and improve its performance by proposing a diverse training pose distribution and a new occlusion mechanism. (3) We extend DeepIM to pose tracking and demonstrate robust performance in both dataset and the real world. (4) We experiment with applying DeepIM on augmented reality and robot link pose tracking.

Reproduction

To reproduce the performance on LINEMOD dataset, we work on Pytorch paszke2017automatic implementation of DeepIM. In the training procedure, real image provides labeling in real world for the network while synthetic data augments the training data and help the network to avoid overfitting. For one epoch, we expect to see one sixth of the real images while the rest are synthetic. In terms of the video split, we sample around 1000 images from the first 6 videos containing the object for training, and leave the rest videos for testing. Different from the original work, we remove the need for mask and flow supervision and render the synthetic data on the fly. Moreover, we experiment with removing the mask in the input since the rendered color image already provides necessary information for the target object. Finally we use a separate regressor for each object instead of a shared “head”, as in the original paper for unseen object generalization. The overall reproduced performance on LINEMOD for both one-class and multi-class is comparable to the original version implemented in MXNet [13].

Synthetic Pose Distribution

We focus on synthetic training and zero-shot sim-to-real transfer of the network. During the experiments, we found that the distribution of the training poses of the synthetic image heavily affect the generalization of the model. We showcase two orientation distributions where “Perturb” denotes the sample ground truth poses with perturbations and “Random” denotes uniformly sampled poses. The expected rotation distribution should cover a sphere if we are rotating a vector in z direction with unit length. However, the perturbation poses only cover a small corner of the sphere. In classical machine learning, the goal of training is to minimize empirical risks, which works well if the test distribution is similar to the training. Therefore, one should start with uniform pose distributions to avoid overfitting. Given that the “Perturb” distribution, noting a concentration on half sphere that represents upright pose, comes from real-world data, we could use it for finetune purposes.

Table 2.1: Comparison between Different Training Pose Distribution

Metric	$5^\circ, 5cm$	Pose	Projection
Perturb	0.5432	0.6391	0.4427
Random	0.8839	0.9793	0.6328

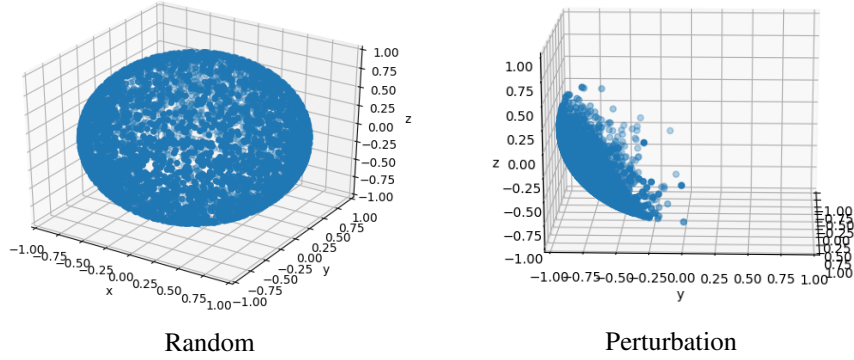


Figure 2.4: Pose Distribution Visualization

Occlusions

To simulate the realistic multi-object occlusion, we render multiple objects with random pose distribution and random backgrounds. A naive occlusion generation is to pick an occluder and place it near the center of the target object through some occlusion. This approach can generate occlusion that penetrates the target object and therefore looks distinct from real world occlusions, as shown in “Naive Occlusion” in Figure 2.6. Instead, we propose a new approach with controllable occlusion to generate more natural occluder poses. In Algorithm 1 we use the 3D bounding boxes as a rough estimate of the object mask and generate occlusion poses through PNP method [65]. The overall algorithm runs efficiently and can be adapted to multiple objects.

Tracking

Considering the similarity between pose refinement and object tracking, it is natural to use DeepIM to track objects in videos. Therefore, we conducted an experiment testing DeepIMs ability to track

Algorithm 1: PNP Occlusion (Object BB_{3d} , Occluder BB'_{3d})

 $BB_{2d} \leftarrow$ Smallest bounding box of BB_{3d} projection

 $BB_{2d} \leftarrow$ Sample a random corner of BB_{2d} to occlude

 Match the opposite corner of BB'_{3d} to BB_{2d}

 SolvePNP for BB'_{3d} and BB_{2d}

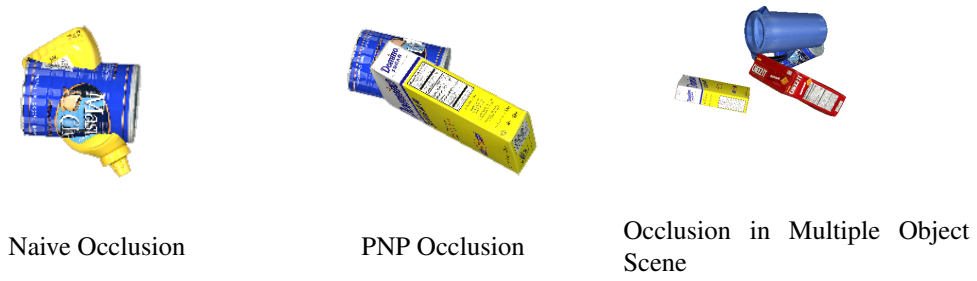


Figure 2.6: Generated Occlusion Images. We can observe that the PNP occlusion look more natural than Naive occlusion.

objects in the YCB-Video dataset. Provided with the ground truth pose of an object in the first frame of each video, DeepIM can perform tracking by using the refined pose estimate from the previous frame as the initial pose of the next frame. Rather than doing inference only on key frames, we applied DeepIM to all images in the test video, in Fig. 2.8, so that the object poses were close between successive frames. Indeed, the transformation between any two steps are not likely to be larger than the perturbation during training. This means that our tracking-by-detection scheme will be minimally affected by the previous drift. In addition, recall that the information and prediction that the network processed stay in image plane, the tracking can work without the actual camera parameters. At inference time, DeepIM network runs at 12 frames per second per object using the NVIDIA 1080 Ti GPU with 2 iterations. The pose tracking is robust with respect to light condition, occlusion, pose initialization and ranges. We tested edge cases such as occlusion center, fast rotation, and farther distance as shown in Figure 2.10. As proposed in [70], a mechanism can be used to determine when DeepIM loses track of an object due to heavy occlusion. A simple strategy is proposed to count the tracking as lost if the last iteration of the last 10 frames has an average rotation

greater than 10 degrees or an average translation greater than 1 cm. Once the tracking is marked as lost, the network will be re-initialized with PoseCNNs prediction. This strategy is designed with the intuition that successful tracking should have a small offset at the last iteration.

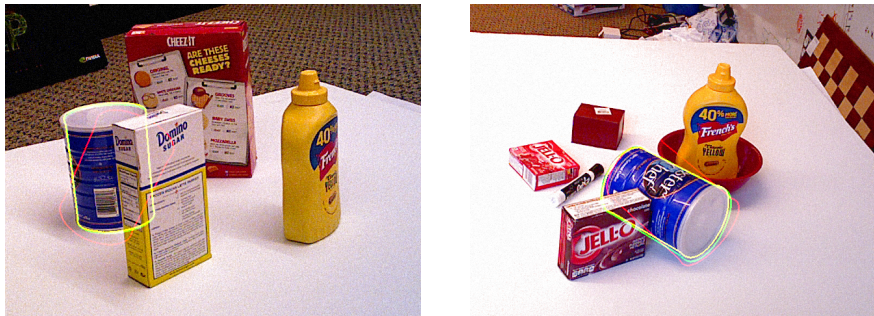


Figure 2.8: Refinement Performance on YCB Video Dataset.

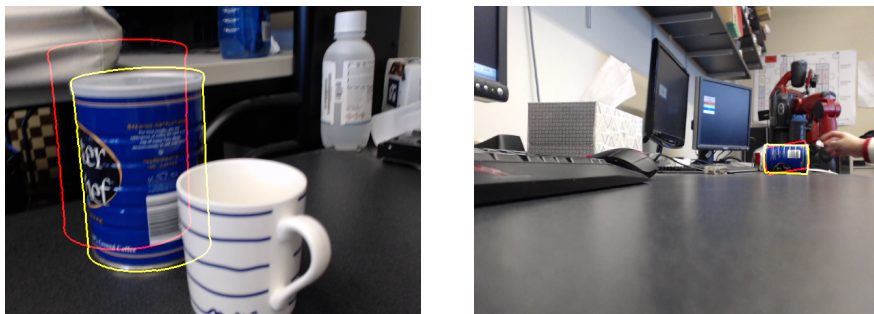


Figure 2.10: Robust Refinement Performance in the Real World.

More Applications

We further showcase the potentials of DeepIM for augmented reality (AR) and robot hand pose tracking. Object pose estimation is tightly related to augmented reality. Given an accurate pose tracking, it's not difficult to render any shapes in a predefined relative pose to the target. A natural object to track in robotics manipulation would be the robot hand, which allows us to estimate both object and robot states, shown in Figure 2.12. An interesting future direction is to investigate if

a relative transform between object and robot hand can be learned by explicitly modeling their relationships as in [104].

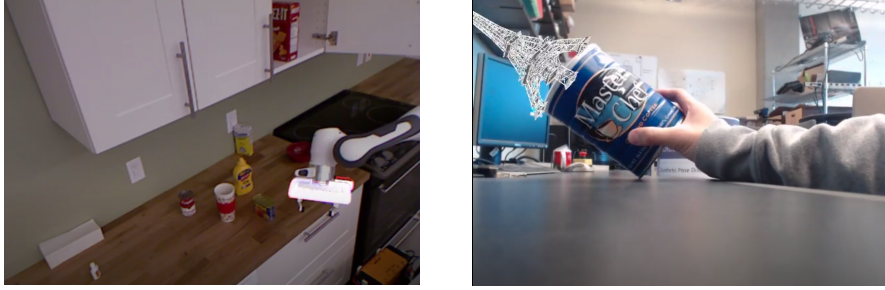


Figure 2.12: DeepIM Application on Robot Pose Tracking and Augmented Reality.

2.4 Experiments

Overall, we conduct ablation study on our improvements. We use the following metrics to evaluate success. The $5^\circ, 5cm$ metric considers an estimated pose to be correct if its rotation error is within 5° and the translation error is below 5cm. *Pose* uses the average distance metric to compute the averaged distance between points transformed using the estimated pose and the ground truth pose. A pose estimation is considered to be correct if the computed average distance is within 10% of the model diameter. *Projection* focuses on the matching of pose estimation on images. We compute the error and accept a pose estimation when the 2D projection error is smaller than a threshold of 5 pixels.

Across experiments, we use PoseCNN [132] as a pose initialization method on YCB-Video dataset. From Table 2.1, we observe a consistent improvement to diversify the training pose distribution. Based on Table 2.2 and Figure 2.14, we also empirically show that tracking improve pose estimate performance than re-initialization in each frame in most cases. The overall DeepIM [70] significantly outperforms state-of-the-art 6D pose estimation methods using color images only and provides performance close to methods that use depth images for pose refinement, such as using the iterative closest point algorithm. However, there are failure cases for the network such as (1) discrepancy between object models and images. (2) few patterns to match. (3) the initial pose is too far away from the correct pose.

Method	Tracking			Reinitialize			PoseCNN		
Metric	$5^\circ, 5cm$	Pose	Projection	$5^\circ, 5cm$	Pose	Projection	$5^\circ, 5cm$	Pose	Projection
Master Chef Can	0.369	0.771	0.231	0.379	0.750	0.254	0.042	0.102	0.001
Sugar Box	0.564	0.745	0.441	0.418	0.581	0.258	0.1362	0.443	0.071
Potted Meat Can	0.402	0.866	0.606	0.375	0.813	0.588	0.106	0.427	0.418
Mustard Bottle	0.799	0.983	0.647	0.811	0.971	0.658	0.193	0.764	0.064
Power Drill	0.371	0.786	0.094	0.276	0.756	0.085	0.089	0.420	0.033
Cracker Box	0.453	0.812	0.241	0.391	0.498	0.074	0.027	0.284	0.001
Mean	0.493	0.827	0.376	0.391	0.441	0.319	0.098	0.406	0.098

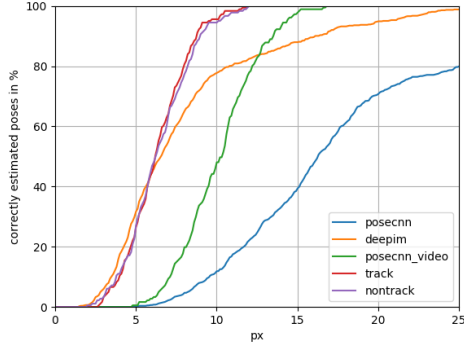
Table 2.2: DeepIM Ablation Study Performance. *PoseCNN* [132] refers to initialize pose estimate with PoseCNN. *Reinitialize* refers to single frame pose refinement by DeepIM. *Tracking* refers to initialize pose estimate with the refined pose from DeepIM.

2.5 Conclusion

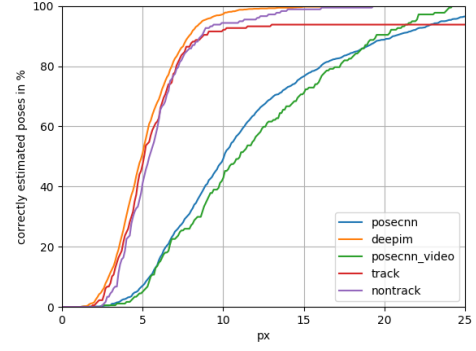
In this work, we study a pose refinement method based on iteratively matching two images. Throughout training, the network is capable of predicting a relative translation and rotation from an initial pose estimate that can come from various pose networks. We conduct follow-up experiments on its extension in pose tracking on YCB dataset. Overall, this learning framework for estimating and tracking pose provides accurate pose information for known objects at real time. The information extracted from perception module is then fed into the planning stage of robot manipulation. In [70], a generalization to unseen objects and a combination with depth information are considered and experiments are more thoroughly conducted to evaluate the success of pose tracking. We refer interested readers to the original work for more ablation study and analysis. For the problem of pose estimation, many current works now focus on lifelong learning, unseen objects, symmetric objects, and systematic way of fusing RGBD information.

2.6 Note

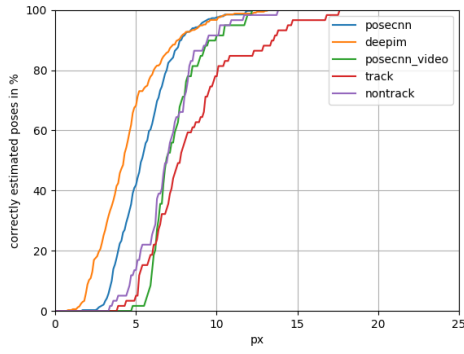
This collaboration work has many shared content with the original work [70] appeared in peer-review publication.



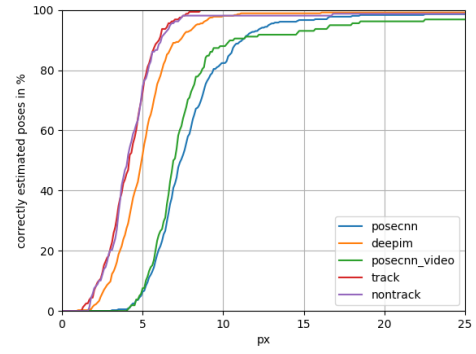
Master Chef Can



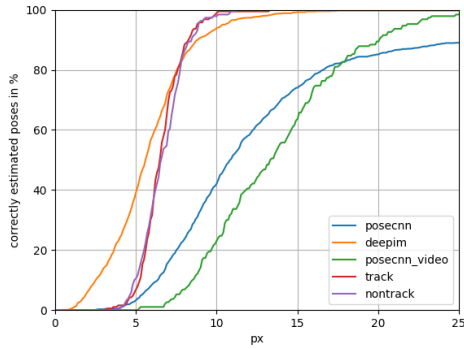
Sugar Box



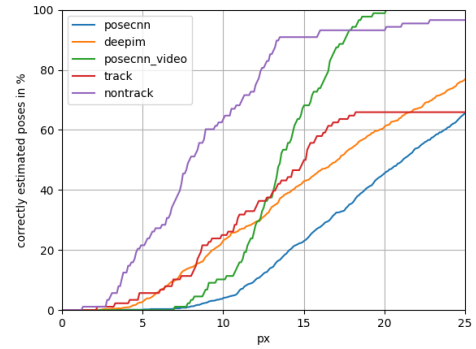
Potted Meat Can



Mustard Bottle



Power Drill



Cracker Box

Figure 2.14: YCB Objects Projection Performance Plot. *deepim* and *posecnn* refer to keyframe estimation in YCB-Video Dataset. We observe an expected improvement through iterative refinements. The rest are tracking for each frame in YCB-Video. We observe a stable performance of pose tracking using DeepIM.

Chapter 3

OPTIMIZATION AND LEARNING FOR MANIPULATION PLANNING

3.1 Introduction

In order to manipulate objects in a cluttered scene, a robot needs to plan its motion to reach and grasp a target object in the scene. Due to the complexity of this manipulation planning problem, it is usually decomposed into two sub-problems and tackled separately: arm motion planning and grasp planning. While arm motion planning aims at reaching the target goal and avoiding obstacles in the scene, grasp planning generates feasible grasps of the target object based on its properties such as its 3D shape and material. However, separating motion planning and grasp planning requires some post-processing steps to connect the two plans. For instance, a common strategy is to loop over each planned grasp and check if there exists a trajectory to reach the grasp. The post-processing usually involves heuristics and leads to sub-optimal solutions. How to efficiently integrate motion planning and grasp planning remains a challenge.

Solutions to this problem are previously introduced both in the traditional motion planning literature [25, 124] and the recent Task And Motion Planning (TAMP) literature [62, 122]. According to when grasps are generated during planning, these methods can be classified into two categories. Methods in the first category generate a fixed set of grasps offline, and formulate the problem as a grasp selection problem [25, 26]. Generating grasps offline can take advantages of well-designed grasp planning algorithms such as [75]. However, a fixed set of grasps limits the number of grasp options during motion planning. Methods in the second category formulate and solve a joint optimization problem of trajectories and grasps, which generate grasps online during trajectory planning. Online grasp planning explores more grasp options. However, since the space of possible grasps is large and highly non-convex for various objects, approximations are often made to simplify the grasp planning problem. For instance, [124, 122] use spheres to approximate the object shape in optimizing grasps, which limits the quality of the generated grasps.

In this work, to overcome the limitations of existing integrated motion and grasp planning meth-

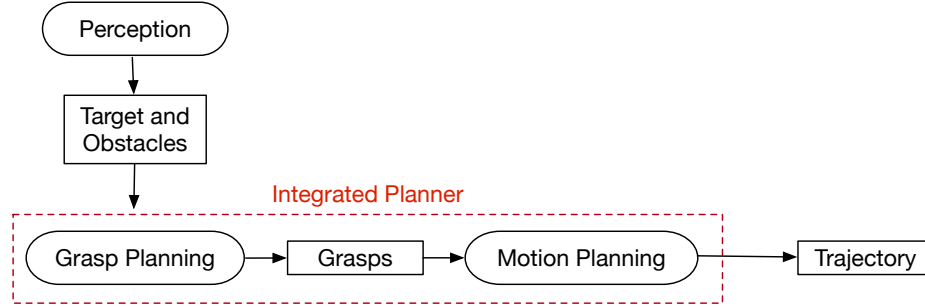


Figure 3.1: Our planning system integrate grasp and motion planning into a single optimization frameworks to output a trajectory for execution. Our method proposes an online goal selection and refinement mechanism, to leverage the goal set information about the target object.

ods [124, 35, 44], we introduce a novel integrated Optimization-based Motion and Grasp Planner (OMG Planner), shown in Fig. 3.1. Our planner combines trajectory optimization with online grasp synthesis and selection. Grasp selection enables us to leverage dedicated grasp planning methods [75, 28] to generate an initial grasp set, which guarantees the quality of the grasps and handles objects with arbitrary shapes. By performing online grasp synthesis, our method augments the initial grasp set and provides more grasp candidates during motion planning. More importantly, the grasp selection problem is formulated as an online learning problem, where a probability distribution of the grasp set is estimated and updated during trajectory optimization, and the best grasp is selected accordingly.

Specifically, our planner optimizes for motion trajectories and grasps jointly. An iterative update algorithm is introduced that alternatively updates trajectories and grasps. In each iteration, given a grasp goal configuration, we utilize the CHOMP trajectory optimization method [98] with a goal set constraint [26] to update the trajectory. To update the grasp, we first assume the availability of a set of grasp candidates that can be obtained from any grasp planning method. Our online grasp selection algorithm then leverages the interplay between the grasp set and the current trajectory to update the probability distribution of the grasp set, from which the optimal goal in the grasp set can be selected. In order to augment the grasp set, we propose a grasp synthesis procedure named Configuration Space Iterative Surface Fitting (C-Space ISF) that refines the grasp configuration of the selected grasp by matching the 3D shapes of the robot gripper and the object. The overall frame-

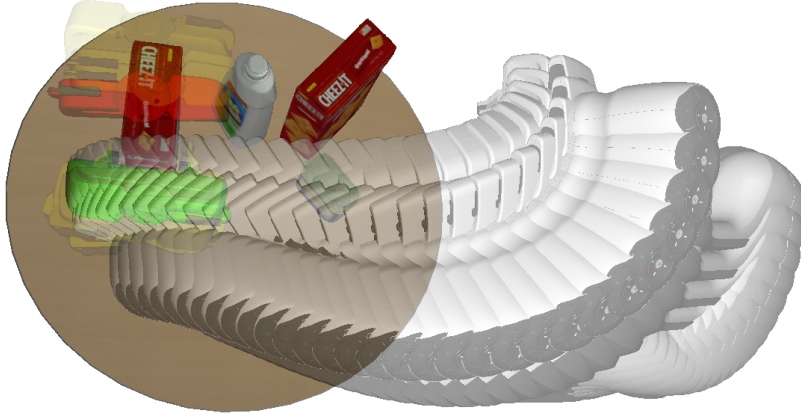


Figure 3.2: A planning scene example. The goal set, initial grasp and selected grasp are yellow, red and green, respectively. Our algorithm synthesizes and selects grasps online and generates motion trajectories to reach the selected grasp.

work leverages the grasp set structure to improve motion and grasp generation. Fig. 3.2 illustrates a planning scene example from our method.

We conduct qualitative and quantitative evaluation to demonstrate the performance of our method for manipulation planning. We also conduct ablation studies on each component of the method. Overall, our contributions are:

1. We introduce an integrated planner that models joint motion and grasp planning as an optimization problem. The computation of feasible and reachable grasps and the search for collision-free trajectories are connected.
2. We propose an online learning algorithm and a grasp synthesis approach to select grasps and generate grasps online, which eliminates the need for a perfect grasp set and grasp selection heuristics.
3. We showcase our algorithm for manipulation planning tasks in cluttered scenes in simulation. By comparing the proposed approach with existing methods, we demonstrate that our integrated planner improves performance.

3.2 Preliminaries and Related Works

3.2.1 Motion Planning

Sampling-based Methods

Sampling-based motion planners such as Rapidly exploring Random Trees (RRTs) [64, 60, 55] and Fast Marching Tree (FMT) [50] find trajectories by incrementally building space filling trees through directed sampling. RRTs offer probabilistic completeness. If there exists a solution to the problem, given sufficient time, a feasible trajectory can be found. However, sampling-based algorithms can be difficult to use in real-time applications due to the computational challenges and requirements for post-processing steps [56].

Trajectory Optimization

Trajectory optimization starts with a possibly infeasible trajectory and then optimizes the trajectory by minimizing a cost function. CHOMP [98] and related methods [26, 25] optimize a cost functional using covariant gradient descent, while STOMP [53] uses stochastic sampling of noisy trajectories to optimize non-differentiable costs. More recently, TrajOpt [106] solves a sequential quadratic program and performs convex continuous-time collision checking. Trajectory optimization methods are fast, but can only find locally optimal solutions. The local nature of previous methods can often be improved by multiple initialization and goals [106, 25].

Our method is based on CHOMP [98] for trajectory optimization. Instead of optimizing trajectories on fixed endpoints, we also consider goal selection and synthesis in our planner. While [25] model the goal selection problem as a regression on trajectory attributes through offline training, our method selects the target goal online during planning.

Machine Learning Inference

[74] and [81] respectively formulate continuous trajectory optimization as Reproducing Kernel Hilbert Spaces (RKHSs) and Gaussian Process (GPs) and reach efficient performance in practice. Their methods provide a natural solution to avoid the fine discretization requirement of previous optimization methods, which is the kernel functions that allow interpolation through fewer sup-

port states. While RKHS representation with different kernels permits the larger gradient step than less-flexible representations, GPMP2 dramatically improves planning speed through modeling as Gaussian Process, inferring on factor graph, and solving least square problem efficiently. More generally, one can remove the limits of fixed endpoints and consider motion planning with a feasible goal set. [86] converts goal set trajectory optimization problem into a density estimation and adopts variational bayes method with a GMM model to sample and optimize trajectories.

3.2.2 Grasp Planning

Grasp Synthesis

Given an object, grasp planning or grasp synthesis refers to the problem of finding a stable grasp configuration that gives strong promise on the success of grasping. This problem is inherently difficult due to its infinite set of candidates and variety of shapes. Abundance of approaches have attempted to take disturbance resistance, dexterity, equilibrium and stability into account. Analytic approaches often model the problem as a constrained optimization problem and measuring the grasp quality directly, therefore providing guarantees on force controllability, contact reachability and caging stability. The precise geometric and physical models of the object is often assumed, which is different from data driven approaches that encode human demonstration, perceptual information or semantics. [7] classifies data-driven grasp synthesis approaches into known objects, familiar objects and unknown objects based on priori about the query objects. With the trend of data-driven methods, people have built datasets such as Columbia Grasp Database [40] and challenges such as Amazon Picking Challenge [19]. However, grasp synthesis still remains as an open problem due to the challenges in generality and performance in practice.

Grasp Quality Measure

Before defining various measures, we introduce two forms of closure in grasping. A grasp is in force-closure if any motion of the object is resisted by the contact forces. A stronger condition is form closure, which means the location of the contact points on the object ensures its immobility. Denote the linear mapping from force to wrench to be grasp matrix G and the convex hull of the primitive wrenches to be grasp wrench space P . If the origin of the wrench space lies inside P , then

the grasp is in form closure. We can define a pair of points as antipodal grasp points if and only if the line connecting the contact points lies inside both friction cones [117]. Note that for parallel-jawed gripper, antipodal point grasp guarantees force closure.

In [16], Roa et al. divides the grasp quality measures into those associated with contacts on objects and those associated with hand configurations. The first group takes object's property into account. The grasp quality associated with G can be its minimum singular value $\sigma_{min}(G)$, volume of ellipsoid in the wrench space $\sqrt{\det GG^T}$, and isotropy index $\frac{\sigma_{min}(G)}{\sigma_{max}(G)}$. There also exists measures that are based on geometric relations such as the shape and area of grasp polygon, the distance between centroid of the contact polygon and the object's center of mass, and independent contact regions. Ferrari and Canny [31] considers the largest minimum resisted wrench, which is the distance from origin of the wrench space to the closest facet of convex hull P . Geometrically, the quality is equivalent to the radius of the largest ball centered at the origin of the wrench space and fully inscribed by P .

Analytic Methods

Analytic methods [85, 32, 12] often require full object information to define grasp qualities such as maximal disturbance resistance, force closure and antipodal grasps. For instance, [17] strive to reduce the dimensionality of the search space for robust grasps and use eigengrasps in the Graspit! simulator [75] to plan with the simulated annealing algorithm. Recently, [29] propose to find robust grasp poses by iterative surface fitting that optimizes error metrics of normal alignments and distances as a least squares problem.

Data-driven Methods

Compared to analytic methods, data-driven approaches [39, 7] have gained popularity since they can work on partial information of objects. For determining object grasps from images, [99] assume objects are put on a table top, and a parallel gripper grasps objects from top-down directions. Due to the multi-modality of grasping tasks, [118] sample 6 DOF grasps and train a neural network to predict a quality score. [80] also focus on the diversity of grasp synthesis, which allows and benefits

motion planning with multiple goals. Compared to these grasp planning methods, our method jointly plans arm motion and grasps to reach objects.

3.2.3 *Integrated Motion and Grasp Planning*

The advantage of an integrated planner is that motion planning and grasp planning impose constraints on each other. The closest works to ours are the integrated motion and grasp planners [124, 35, 44]. These methods are built upon sampling-based motion and grasp planners. They bias a motion planner to sample nodes that lead to better grasps while a grasp sampling planner synthesizes grasps online. Different from these methods, we focus on studying the grasp selection problem in a trajectory optimization framework. Therefore, our method can utilize high-quality grasps generated from well-designed grasp planners, such as grasps synthesized from physics simulation [75, 28].

3.2.4 *Task and Motion Planning*

TAMP methods focus on how to combine high-level task planning with low-level motion planning [62, 112, 36, 122], where the tasks usually involve hybrid states and require multiple steps of actions such as picking, placing and pushing. For example, [62] leverage geometric constraints from motion planning to reduce the search space of possible actions. [36] build factored transition systems with sampling-based algorithms for robotic manipulation problems. [122] formulate a constrained path optimization problem for sequential manipulation that involves physical interactions by logic geometric programming [121]. Our approach is related to TAMP for manipulation planning. However, we neither perform any high-level reasoning nor model sequences of actions. We focus on obstacle avoidance with kinematics constraints for grasping. While TAMP methods usually simplify or approximate the geometry of the manipulated objects, we do not make such approximations. Our method can benefit TAMP by providing a joint motion and grasp planner for task-level reasoning.

3.2.5 *Online Learning*

Compared with statistical learning in which data is assumed to come from a distribution and the goal is to minimize an excess risk, online learning instead considers a setting where a learner makes a decision that triggers a loss function online at every time step and the learner needs to dynamically

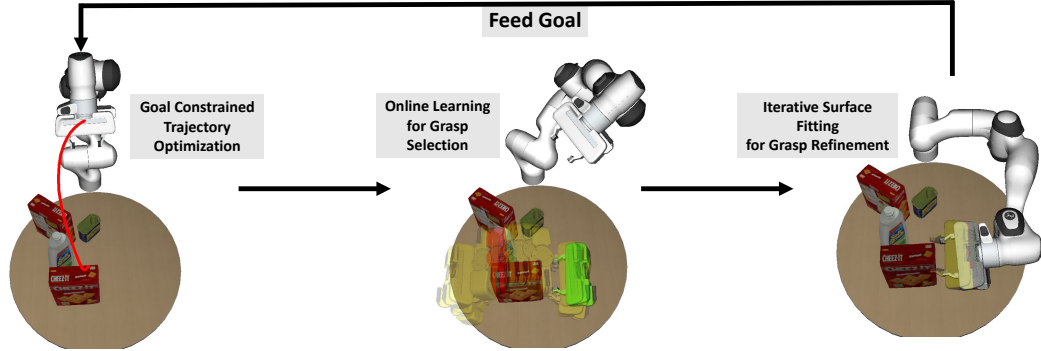


Figure 3.3: Our Optimization-based Motion and Grasp Planner (OMG Planner) consists of three components. The motion planner component optimizes the current trajectory denoted by the red line for a fixed target goal. The online learning component considers the goal set (yellow), and update the initial grasp (red) with a new grasp (green). The grasp refinement component iteratively improves the grasp (green to yellow). The refined grasp is used as the target goal for the next iteration.

improves its decision making by minimizing the accumulated losses [10, 45, 63]. In the regret-based framework, the learner tries to minimize the regret on the loss sequence. Online interactive learning has been successfully applied to tackle different problems in robotics such as manipulation [59], navigation [38], and model predictive control [127]. Here, we formulate the grasp selection problem as a linear bandit in the online learning framework, where our learner (the planner) needs to choose the grasp goal online and adapt to some measured costs during every iteration of the trajectory optimization.

3.3 Methodology

The key idea of our method is to simultaneously optimize the arm motion trajectory and the end-effector configuration for grasping. We formulate the problem as a constrained trajectory optimization problem, where we require the trajectory to avoid obstacles and to be smooth. Meanwhile, the last configuration of the trajectory should afford grasping. To achieve this, our method synthesizes and selects grasps online during trajectory optimization. Fig. 3.3 presents an overview of our optimization-based motion and grasp planner.

Formally, we define a trajectory $\xi : [0, 1] \mapsto \mathcal{Q}$ to be a function that maps time $[0, 1]$ to the robot configuration space $\mathcal{Q}$, where $\mathcal{Q} \subset \mathbb{R}^d$ and d is the degree of freedom of the robot manipulator.

Given a scene of multiple objects, we define the feasible grasp set of a target object as $\mathcal{G} \subset \mathcal{Q}$. Then we solve the following optimization problem to find the optimal trajectory of the robot manipulator for grasping:

$$\begin{aligned} \xi^* &= \underset{\xi}{\operatorname{argmin}} f_{\text{motion}}(\xi) \\ \text{s.t. } \xi(1) &\in \mathcal{G}, \end{aligned} \quad (3.1)$$

where f_{motion} is the objective function of the trajectory, and $\xi(1)$ indicates the last configuration of the trajectory. The constraint in Eq. (3.1) requires the last configuration at time 1 to be a feasible grasping configuration.

3.3.1 Trajectory Objective Functional

Similar to CHOMP [98], we model the objective functional

$$f_{\text{motion}}(\xi) = f_{\text{obstacle}}(\xi) + \lambda f_{\text{prior}}(\xi), \quad (3.2)$$

where the obstacle term f_{obstacle} bends the trajectory away from obstacles by penalizing parts of the robot that are close to or already in collision with objects in the scene, the prior term f_{prior} measures the dynamical quantities across the trajectory such as velocities and accelerations, and λ is a weight to balance the two terms. In this work, the prior term is defined as the integral over squared velocity norms:

$$f_{\text{prior}}(\xi) = \frac{1}{2} \int_0^1 \|\xi'(t)\|^2 dt, \quad (3.3)$$

where $\xi'(t)$ indicates the velocity of the trajectory at time t .

To define the obstacle term, let $\mathcal{B} \subset \mathbb{R}^3$ be a set of body points on the robot and $x(q, u) : \mathcal{Q} \times \mathcal{B} \mapsto \mathbb{R}^3$ the forward kinematics mapping a body point $u \in \mathcal{B}$ with configuration $q \in \mathcal{Q}$ to the workspace. Furthermore, let $c_{\text{obstacle}} : \mathbb{R}^3 \mapsto \mathbb{R}$ be a workspace cost function that penalizes points in the workspace inside and around the obstacles using the Signed Distance Field (SDF). Since we want to drive the body points away from collision, the obstacle term in Eq. (3.2) is an integral that collects the cost of body points on the robot in the workspace along the trajectory:

$$f_{\text{obstacle}}(\xi) = \int_0^1 \int_{\mathcal{B}} c_{\text{obstacle}}(x(\xi(t), u)) \left\| \frac{d}{dt} x(\xi(t), u) \right\| du dt. \quad (3.4)$$

Please refer to [98] for the definition of c_{obstacle} and the derivation of the gradients of f_{prior} and f_{obstacle} .

3.3.2 Iterative Update Rule

In practice, we use a discretization of the trajectory function over time: $\xi \approx (q_1^\top, q_2^\top, \dots, q_n^\top)^\top \in \mathbb{R}^{n \times d}$ for resolution n . Under this parametrization, we can write the prior term as:

$$f_{\text{prior}}(\xi) = \frac{1}{2} \|K\xi + e\|^2 = \frac{1}{2} \xi^\top A \xi + \xi^\top a + c, \quad (3.5)$$

where K is a finite differencing matrix, e is a vector that accounts for the contributions of the configurations that remain constant in the trajectory, i.e., the start configuration and the goal configuration, and $A = K^\top K$ is the dynamic matrix. The covariant idea of CHOMP comes from the inverse of the dynamic matrix. A^{-1} acts as a smoothing operator that propagates the Euclidean gradient of the objective along the trajectory and seeks to make small changes in the average acceleration. Given this parametrization, CHOMP is a variant of gradient descent that minimizes the linear approximation of f_{motion} about ξ within an ellipsoid trust region. The distance metric that shapes this ellipsoid is defined by A , and we obtain the update rule in gradient descent for the i th iteration:

$$\xi_{i+1} = \xi_i - \frac{1}{\eta_{\text{motion}}} A^{-1} v_i, \quad (3.6)$$

where $v_i = \nabla f_{\text{motion}}(\xi_i)$ is the discretized functional gradient and η_{motion} is the step size.

3.3.3 Goal Set Constraint

The end-point of ξ_i can be varied during planning for a set of feasible goal configurations $\mathcal{G}$. Following [26], this is achieved by introducing a constraint on the trajectory. Given a general constraint $h(\xi) = 0$, we can linearize h around ξ_i : $h(\xi) \approx h(\xi_i) + \frac{\partial}{\partial \xi} h(\xi_i)(\xi - \xi_i) = C(\xi - \xi_i) + b$, where C is the Jacobian of h evaluated at ξ_i and $b = h(\xi_i)$. We use Lagrangian to solve the linearized constraint and yield the projected gradient update rule:

$$\begin{aligned} \xi_{i+1} = & \xi_i - \frac{1}{\eta_{\text{motion}}} A^{-1} v_i \\ & + \frac{1}{\eta_{\text{motion}}} A^{-1} C^\top (CA^{-1}C^\top)^{-1} CA^{-1} v_i \\ & - A^{-1} C^\top (CA^{-1}C^\top)^{-1} b. \end{aligned} \quad (3.7)$$

Assume our goal configuration is g at the i th iteration. We simply model the goal set constraint as

$$h(\xi_i) = \xi_i(1) - g = 0, \quad (3.8)$$

and denote the update rule in Eq. (3.7) by $CHOMP-Proj(\xi, g)$. With this update rule, we denote $f_g(\xi_i)$ as the objective cost if we use g as the goal from the i th iteration. As we move the end-point, CHOMP smoothly updates the remaining trajectory. This formulation is different from the vanilla CHOMP in that the goal is not fixed and can change in different iterations.

3.3.4 Online Learning for Grasp Selection

While [26] address the problem of CHOMP planning with a goal set constraint, modeling goal selection as projection by Eq. (3.8) can often be simplistic. [25] propose a few trajectory attributes such as goal radius, elbow room, distance, etc., for a regression formulation. Indeed, the optimality of a goal should be induced by the objective function and thus related to the success of motion planning. Given that we can optimize $f_g(\xi_i)$ with a target goal, we propose to choose the goal that maximizes our motion generation success under an online linear bandit scheme [10, 63]. An online linear optimization framework is suitable because at every iteration, we can model the objective costs of the goal set as our suffered losses and relax our goal selection as probability distributions.

Assuming the goals in $\mathcal{G}$ are intricately correlated, our strategy should dynamically choose the goal for the next iteration until it finds the best goal. In CHOMP projection, we suffer from switching to goals that can heavily perturb our current trajectory into collision. Therefore, to allow for principled behavior that selects the next goal g_{i+1} , we need a strategy that balances exploitation and exploration of the costs.

In the i th iteration, denote the goal distribution on a discrete goal set $\mathcal{G}$ by p_i . We apply online learning to solve for p_{i+1} . To use optimization for goal selection, we need to define the cost of each goal in the goal set, denoted by a cost vector c_i . Ideally, we would like to have $c_i(g) = f_g(\xi_i)$, then the objective cost directly tells us which target goal to use. However, since $f_g(\xi_i)$ is not available without running CHOMP for each goal, we generate surrogates to our objective cost $f(\hat{\xi}_i^g) \approx f_g(\xi_i)$ by estimating the potentially optimized trajectory $\hat{\xi}_i^g$ as if the trajectory ξ_i ends at $g \in \mathcal{G}$.

We propose that to compute $f(\hat{\xi}_i^g)$, it is more natural to look at the tail of trajectory denoted by $\tilde{\xi}_i^g$ starting at $t_i = \frac{i}{N}$, where N is our optimization horizon. To motivate this, recall that in

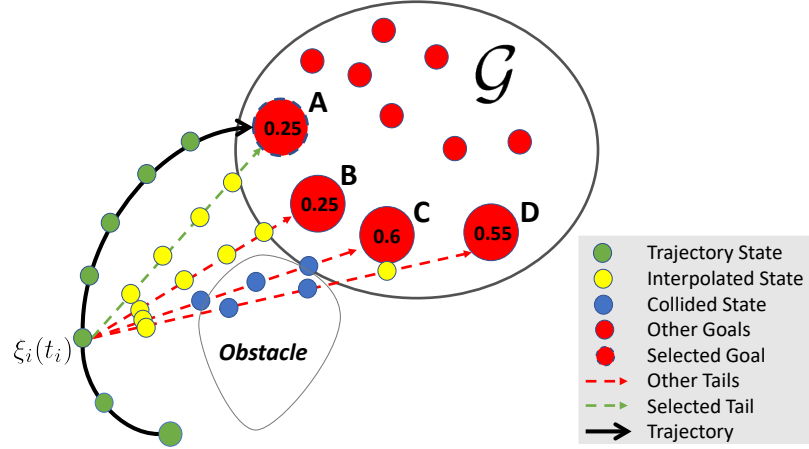


Figure 3.4: Online goal selection algorithm at the i th iteration. The goal costs $c(g)$ are denoted inside red circles. In this example, the online learning algorithm is about to shift goal from A to B , which yields a collision-free yet shorter trajectory.

each *CHOMP-Proj* step, we project onto the goal constraint and propagate the changes smoothly. Therefore, as the time step i approaches N , the changes to the trajectory impact more on the tail than the head of the trajectory. Our estimated costs would be more accurate if we use the tail $\tilde{\xi}_i^g$ instead of the whole trajectory. Moreover, the goal cost vector needs to converge for stability of the algorithm and it should also be dynamic from the learning perspective. Another way to think about this is that we are “simulating” the optimal control execution of the trajectory during planning and estimating the running cost for dynamic goal selection. As shown in Fig. 3.4, since the prior of CHOMP is the minimum smoothness trajectory, one can approximate the trajectory tail $\tilde{\xi}_i^g$ as the interpolation between $\xi_i(t_i)$ and the goal g with constant velocity. The associated cost for the next iteration is defined as $c_{i+1}(g) = f(\tilde{\xi}_i^g)$ and normalized to have unit norm.

Given our estimated objective cost $c_{i+1} = \{f(\tilde{\xi}_i^g) \mid g \in \mathcal{G}\}$ and previous costs $c_1, \dots, c_i$ for the goal set $\mathcal{G}$, we apply online learning algorithms to update the goal distribution p_{i+1} as described later. Then the expected loss at the $(i+1)$ th iteration under this distribution is $\langle c_{i+1}, p_{i+1} \rangle$ and g_{i+1} is defined as the mode of the distribution p_{i+1} , which turns out to be more stable than a sample. Overall, online learning for goal selection aims to minimize the regret

$$\mathcal{R}_N = \sum_{i=1}^N \langle c_i, p_i \rangle - \min_{g \in \mathcal{G}} \sum_{i=1}^N c_i(g), \quad (3.9)$$

which compares the loss of our strategy p_i up to time N with the best cost of any fixed goal in hindsight. In the following, a few commonly used optimization methods for online learning problems [10] are presented. These methods can be used to solve p_{i+1} at the i th iteration.

Follow the Cheapest (FTC)

FTC greedily moves the distribution p_{i+1} entirely to the point that has the minimum cost $c_{i+1}(g)$. *CHOMP-Proj* turns out to be a FTC algorithm with the costs defined by the distances between the end-point of the current trajectory and the goals.

Follow the Leader (FTL)

FTL computes the point $g \in \mathcal{G}$ that causes the lowest total cost $\sum_{j=1}^{i+1} c_j(g)$ and then makes the distribution p_{i+1} concentrated entirely on g .

Exponential Weighting (EXP)

EXP begins with an uniform distribution over goals: $p_1(g) = \frac{1}{|\mathcal{G}|}, \forall g \in \mathcal{G}$. At time i , it first multiplies the previous probabilities with the corresponding exponential costs: $\hat{p}_{i+1}(g) = \exp(-\eta_{ol} c_{i+1}(g)) p_i(g), \forall g \in \mathcal{G}$, where η_{ol} is the learning rate. Then it normalizes the vector to make it a probability distribution: $p_{i+1}(g) = \hat{p}_{i+1}(g) / \sum_{g \in \mathcal{G}} \hat{p}_{i+1}(g)$.

Mirror Descent (MD)

We refer the readers to [5, 63] for this form of regularized optimization in a bandit setting. To allow MD to prefer probability distributions that are more diversified, we associate the Bregman divergence with entropy, which becomes the KL divergence. At time step i , the MD update rule is $p_{i+1} = \underset{p \in \Delta}{\operatorname{argmin}} \eta_{ol} \langle c_{i+1}, p \rangle + D_{KL}(p || p_i)$, where Δ is the probability simplex and η_{ol} is the learning rate. The argmin of this update rule can be solved by Lagrangian.

3.3.5 Online Grasp Synthesis

So far we have assumed that the goal set is given and fixed. A fixed set of grasps limits the number of grasp possibilities. Even with a set of perfect grasps in the workspace, it is unclear that solving

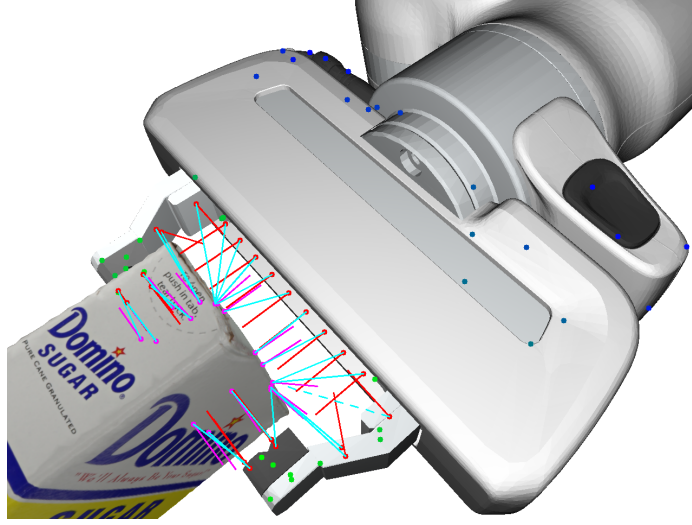


Figure 3.5: Red points denote the sampled contact points u_h with normals u_h^n and purple points denote the corresponding object points u_o with normals u_o^n . The correspondence lines are colored in cyan. Body points u range from blue to green, denoting from far to close with respect to objects.

inverse kinematics would give a sufficient goal set in the configuration space. A fixed set also has low adaptability to the environment since it fails to explore new grasps.

Starting from an initial finite goal set $\mathcal{G}$, we propose to synthesize more candidate grasp configurations online. Suppose after the online learning process for goal selection, the configuration end goal g_i is selected at iteration i . For simplicity, we drop the iteration index i in the followings. Our proposed grasp synthesis algorithm aims to improve the grasp affordance by minimizing the objective function:

$$f_{\text{grasp}}(g) = f_{\text{isf}}(g) + \gamma f_{\text{collision}}(g), \quad (3.10)$$

where $f_{\text{isf}}(g)$ measures the grasp quality using the Iterative Surface Fitting (ISF) algorithm, $f_{\text{collision}}(g)$ measures collisions in the scene, and γ is a constant to balance the two terms. We use $f_{\text{isf}}(g)$ to refine the end configuration g such that the end effector transformation $T \in \mathbb{SE}(3)$ makes the surface of the robot gripper $\mathcal{S}$ match against the surface of the target object $\mathcal{O}$. We softly penalize hand and arm collisions and use gradient descent to optimize the objective. This enables efficient and stable optimization in the configuration space.

Specifically, we generalize the forward kinematics mapping $x(g, u)$ of a body position u in Sec. 3.3.1 to include directions: $x(g, u^n)$ maps a body direction vector u^n and a robot configuration

g to a direction vector in the workspace. Observe that the end effector transformation T in the workspace is the hand coordinate transformed by g . We can write all workspace costs on T as the configuration space costs on g . We define a set of hand contact points $\{h_j\}$ and a set of contact normals $\{h_j^n\}$ on the robot hand and fingers, indexed by j (see Fig. 3.5). Denote $u_h = \{x(g, h_j)\}$ and $u_h^n = \{x(g, h_j^n)\}$ as the two sets in the workspace transformed by forward kinematics. We then search for the nearest neighbor point u_o and normal u_o^n on object $\mathcal{O}$ associated with u_h, u_h^n . For m pairs of points and normals, we define the point matching loss as

$$f_{\text{pml}}(g) = \sum_{j=1}^m \langle u_{h,j} - u_{o,j}, u_{o,j}^n \rangle^2, \quad (3.11)$$

and the normal alignment loss as

$$f_{\text{align}}(g) = \sum_{j=1}^m (\langle u_{h,j}^n, u_{o,j}^n \rangle + 1)^2. \quad (3.12)$$

The point matching loss closely attaches the hand to the object, and the normal alignment loss aligns their surface normals. Overall, the ISF loss is defined as

$$f_{\text{isf}}(g) = f_{\text{pml}}(g) + \alpha f_{\text{align}}(g), \quad (3.13)$$

where α is a weight coefficient.

Since the hand has to remain collision-free with respect to the object, we evaluate the object points in the SDF of the robot hand and penalize any hand collision $f_{\text{hand_obstacle}}(g)$. This is different from the obstacle cost in CHOMP, where robot body points are evaluated in the SDFs of objects. Note that the gradient for object points in the SDF of the hand can be treated as hand points pushing in opposite gradient directions. We also add the CHOMP obstacle cost evaluated at a single configuration g , $f_{\text{obstacle}}(g)$, to allow local arm obstacle avoidance. The overall collision cost for grasp synthesis is

$$f_{\text{collision}}(g) = f_{\text{hand_obstacle}}(g) + \beta f_{\text{obstacle}}(g), \quad (3.14)$$

where β is a balancing weight.

Algorithm 1 presents our C-Space ISF algorithm for grasp refinement. The workspace cost f_{grasp} in Eq. (3.10) and its gradient ∇f_{grasp} are computed by the *Error* function. The gradient ∇f_{grasp} is updated via the exponential map *exp* from $se(3)$ to $\mathbb{SE}(3)$ and pulled back to the configuration

space ∇g through the Jacobian transpose $J^\top$ with a step size η_{grasp} . We then update the grasp g in $\mathcal{G}$ and return back to motion planning step. In practice, one can run the C-Space ISF algorithm 2 multiple times for each motion planning iteration.

Algorithm 2: C-Space ISF ($g_i, u_h, u_h^n, \mathcal{O}, \mathcal{S}$)

$$u_o, u_o^n = \text{NearestNeighbor}(u_h, u_h^n, \partial\mathcal{O})$$

$$f_{\text{grasp}}, \nabla f_{\text{grasp}} = \text{Error}(u_h, u_h^n, u_o, u_o^n, \mathcal{O}, \mathcal{S})$$

$$\nabla g_i = J^\top \exp(\nabla f_{\text{grasp}})$$

$$g_i = g_i - \eta_{\text{grasp}} \nabla g_i$$

3.3.6 Joint Motion and Grasp Planning

We have defined our iterative update rule for the i th iteration, involving optimizations of smoothness, collision, goal selection, and grasp quality. Our integrated planner simply runs this iterative update rule for N times. A pre-termination criterion can be set with the threshold on grasp quality, trajectory smoothness, and collision-free requirement. Our joint planning algorithm is shown in Algorithm 3. *CHOMP-Proj* provides a way to smoothly update the trajectory to avoid obstacles given a goal. The *ONLINE-LEARNING* as described in Sec. 3.3.4 considers all grasps in the goal set and updates our goal distribution for the next iteration. The *C-Space ISF* applies to the selected goal to refine its quality.

It is worth to mention that we tried to formulate a joint optimization problem by combining the two objective functions in Eq. (3.1) and Eq. (3.10). However, we found that the ISF algorithm heavily depends on the initialization and is often stuck in local optima with bad initializations. Our current formulation provides good initialization to the ISF algorithm using the goal set, which makes the optimization more stable.

3.4 Experiments

We conduct experiments with the Panda Franka arm, a 7-DOF arm with a parallel gripper (Fig. 3.3). We generate cluttered scenes by dropping sampled YCB objects [11] with random upright poses on a table in the Pybullet Simulator [20]. Each scene samples 3 to 7 objects. The objects used are

Algorithm 3: OMG Planner

```

initial trajectory  $\xi_0, \mathcal{O}, \mathcal{G}, \mathcal{S}, p_0, g_0$ 

for  $i = 0, \dots, N$  do
     $\xi_{i+1} = \text{CHOMP-Proj}(\xi_i, g_i)$ 
     $p_{i+1} = \text{ONLINE-LEARNING}(\xi_{i+1}, \mathcal{G})$ 
     $g_{i+1} = \text{argmax}(p_{i+1})$ 
     $g_{i+1} = \text{C-Space ISF}(g_{i+1}, \mathcal{O}, \mathcal{S})$ 
end

```



Figure 3.6: Three initial grasp sets of the mustard bottle.

master chef can, cracker box, sugar box, tomato soup can, mustard bottle, potted meat can, pitcher base, bleach cleanser, bowl, and mug.

We experiment with three different sets of initial grasps as shown in Fig. 3.6. The “Simulated” grasps are sampled from a physics-based simulator [28]. The second set is sampled from the Graspit! planner [17]. The “Approach” grasps are collision-free grasps naively sampled by random approach directions. The initial start configuration is fixed, and goal configurations are generated from inverse kinematics and checked collision-free. A trajectory is then initialized as a cubic spline connecting the start configuration and the initial goal.

3.4.1 Evaluation Metrics

Motion planning methods are usually evaluated by the feasibility of the generated paths, i.e., collision-free paths. In addition, we measure several attributes of the paths that are desirable in our grasping

tasks.

Trajectories close to obstacles should be avoided in practice. We measure “collision” as the sum of obstacle costs c_{obstacle} of sampled points inside the minimum padding distance (5cm) by densely interpolating each trajectory to contain 200 time steps. The smoothness of a trajectory is another desired property. We measure “smoothness” by the squared velocity norm as f_{prior} . To measure the planning performance, we define the “planning” success as a collision-free path with its smoothness below a threshold of 30.

After planning, it is important to see how the plan is executed for grasping. The success of grasp execution involves system dynamics, feedback controllers and grasp robustness, in addition to trajectory collision and smoothness. We execute a plan in the Pybullet simulator and define the “execution” success if the target object is lifted. We also compute the “grasp” cost $f_{\text{grasp}}(\xi(1))$ from our C-Space ISF method to measure the grasp quality. Finally, planning “time” is measured in seconds for different algorithms.

3.4.2 Implementation Details

CHOMP

We set the smooth weight $\lambda = 0.1$, discretization resolution $n = 30$, and learning rate $\eta_{\text{motion}} = 0.01$. The obstacle padding is 0.2m within which the cost increases quadratically [98]. A collision-free path is assumed to have a minimum distance of 5cm from obstacles. Instead of modeling the arm with swept spheres as in the original CHOMP, we uniformly sample body points on the surface of the robot, and select 500 points in the trajectory with the largest obstacle costs during optimization to update the obstacle gradient (see Fig. 3.5). These modifications make the integral over $\mathcal{B}$ more general and precise as well as stabilize the optimization. We sample 20 points from each body part in our implementation. Since grasping tasks require collision-free gripper poses, we also include sampled points from hand and fingers.

Online Learning

The EXP algorithm uses the learning rate $\eta_{\text{ol}} = \sqrt{\log(|\mathcal{G}|)/N}$. The MD algorithm takes the learning rate over an ensemble of experts with $\eta_{\text{ol}} = 2^k \log(N)$ and k is selected from $\{-7, -4, 0, 1, 3\}$.

100 scenes, 30 grasps each scene, 10 runs						
Algorithm	Fixed	Proj	FTC	FTL	EXP	MD
Execution	74%	75%	81%	79%	80%	81%
Planning	78%	79%	91%	89%	90%	92%
Smoothness	18.87	11.24	15.78	15.86	15.71	15.54
Collision	5.97	12.06	4.68	4.10	4.95	3.87
Time	0.18	0.15	0.20	0.22	0.23	0.30

Table 3.1: Comparison between two baselines and four online learning algorithms for grasp selection.

C-Space ISF

We set the normal penalty weight $\alpha = 0.01$, the arm collision coefficient $\beta = 0.001$, the hand collision constant $\gamma = 0.5$, and the learning rate $\eta_{\text{grasp}} = 0.05$. The nearest neighbors are found with a KD-Tree on 1000 sampled points for each object, where repeated neighbors are filtered out to increase robustness.

3.4.3 Grasp Selection

We first evaluate different algorithms for grasp selection in our trajectory optimization framework. In this experiment, the “Simulated” grasp set is used and no grasp refinement is performed. Therefore, the dominating factor in planning performance is the goal selection strategy. Table 3.1 presents the results for 100 scenes, where we compare the four online learning algorithms with two baselines. 1) “Fixed” selects the best goal based on the initial objective estimate and fixes the goal during planning. 2) “Proj” [26] uses the *CHOMP-Proj* update rule which always projects to the closest goal during trajectory optimization.

Our results in Table 3.1 indicate that the online learning methods improve the success of planning and execution. From the improvement over “Fixed”, it is clear that planning toward a fixed goal does not provide the same performance as online algorithms that adaptively select goals, since the initial goal may be sub-optimal. Comparing the four online learning algorithms with “Proj”, we can see that the cost estimate based on our objective function performs better than just using the distance between the trajectory end-point and grasps in the goal set. In addition, modeling the probability

100 scenes, 60 grasps, 30 iteration refinements, 10 runs						
Object Name	S	S Refined	G	G Refined	A	A Refined
Cracker Box	2.47	2.32	4.27	3.31	6.58	4.36
Mug	5.11	3.93	7.24	5.55	9.05	5.18
Tomato Soup Can	2.38	2.27	3.15	2.83	7.06	6.27
Mustard Bottle	4.40	3.19	4.91	3.79	8.15	5.73
Potted Meat Can	2.96	1.95	3.08	2.54	7.47	4.91
Pitcher Base	4.41	2.97	5.68	3.76	8.69	6.14
Mean	3.62	2.77	4.72	3.63	7.83	5.43
Planning	92%	93%	84%	84%	71%	73%
Execution	80%	82%	56%	59%	26%	33%

Table 3.2: Grasp costs for six YCB objects before and after refinement. “S”, “G” and “A” denotes the simulated set, the grasps set, and the approach set, respectively. Planning and execution success are from 10 runs of the 100 planning scenes.

distribution of the goal set for grasp selection is beneficial. Our empirical analysis shows that these online learning methods provide a scheduling behavior for a planner to adaptively switch goals while keeping optimization stability. Finally, MD performs slightly better than the others with a trade-off in running time.

3.4.4 Grasp Refinement

We investigate the performance of the C-Space ISF algorithm for grasp refinement. In this experiment, we apply the MD algorithm for grasp selection and test the C-Space ISF algorithm on the three different grasp sets (Fig. 3.6). Table 3.2 shows the detailed grasp costs and the planning and execution success before and after grasp refinement.

Overall, we observe a consistent improvement on different initial grasps from grasp refinement. The “Simulated” set achieves the minimum grasp cost and the highest success rate. Because these grasps are sampled and verified in simulation by grasping and shaking an object to see if the object falls or not [28]. The drawback is that it is time-consuming to generate these high-quality grasps. Our refinement algorithm can further improve the quality of these simulated grasps. We can also see that the C-Space ISF algorithm heavily depends on the initial grasps. Therefore, the improvement

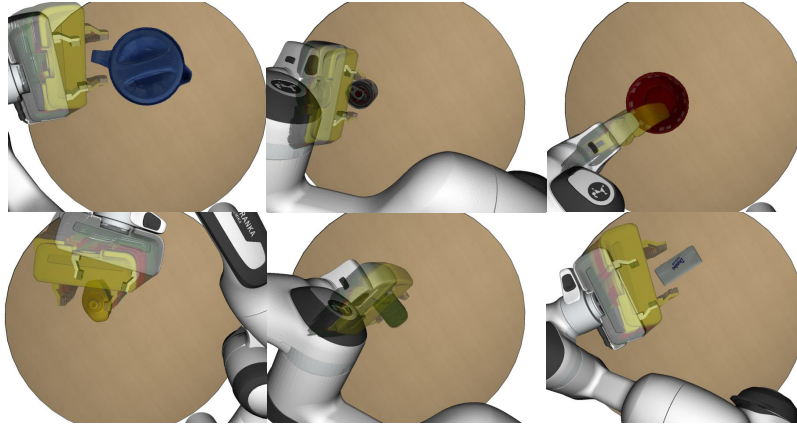


Figure 3.7: Grasp optimization results. The intermediate grasps are colored from green to red and yellow.

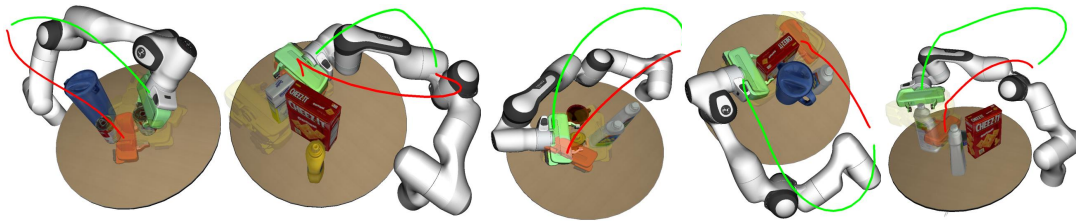


Figure 3.8: Planning scene examples. Red trajectories come from CHOMP, and green trajectories come from our OMG Planner.

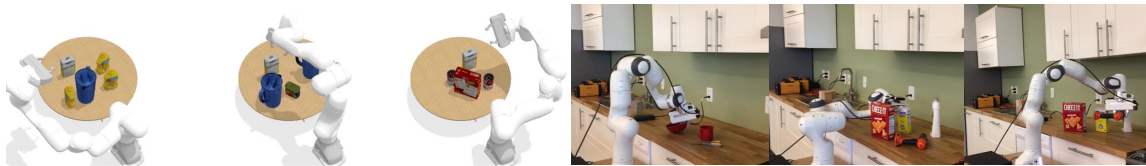


Figure 3.9: Left) Planning scene examples in Pybullet. Right) Successful real world grasping examples by executing trajectories from our planner using estimated 6D object poses from [22]

100 scenes, 30 grasps each scene, 10 runs					
Algorithm	CHOMP	CHOMP-C	OMPL-FMT	OMPL-RRT	OMG
Execution	77%	75%	66%	64%	84%
Planning	82%	79%	80%	78%	93%
Smoothness	19.12	11.24	27.21	26.45	15.43
Collision	4.78	12.06	6.37	6.52	3.92
Grasp	3.43	3.35	3.49	3.48	3.04
Time	0.25	0.15	0.67	0.22	0.42

Table 3.3: Comparison on planning methods for joint motion and grasp planning. CHOMP [98], CHOMP-C [26], FMT [50], RRT-Connect [60].

on the “Graspit” set and the “Apporach” set is limited. This is also the reason why we combine grasp selection with grasp refinement, instead of optimizing grasps from random initial grasps. Fig. 3.7 shows some qualitative examples of our grasp refinement method.

3.4.5 Planner Performance

We compare our OMG planner with other motion planners in the literature, where the “Simulated” grasp set is used. The OMG planner uses the MD algorithm for grasp selection and the C-Space ISF algorithm for grasp refinement. The results are presented in Table 3.3. Since grasp planning is not covered by CHOMP [98], FMT [50] and RRT-Connect [60], we adopt the common routine used in practice to generate manipulation trajectories for them: 1) ranking the goals in the grasp set based on heuristics such as our objective estimates; 2) planning through the goal set until a feasible trajectory is found. We perform planning experiments for 100 scenes with 30 initial grasps each scene. The numbers in Table 3.3 are averaged over 10 runs for each scene.

Comparing OMG with other planners, we can see the improvement on planning and execution thanks to our grasp selection and refinement process. By modeling a probability distribution on the grasp set, our method globally selects the optimal grasp for planning. In our metrics, planning success acts as a feasibility and optimality of the trajectory but execution involves control and system noises. By terminating at the first feasible trajectory, other methods can generate sub-optimal plans to execute. In contrast, with the option to select goals in the goal set, our trajectory optimization

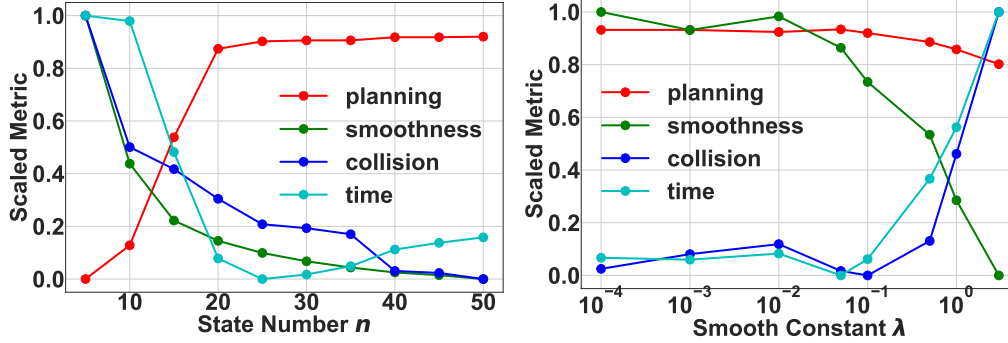


Figure 3.11: Ablation study of n and λ . The metrics (except planning) are scaled between 0 and 1. Left) We see a decrease of smoothness and a increase of success rate due to finer representation. The planning time drops first due to more solvable problems and then increases due to increase of computation. Right) The smoothness weight λ around 0.1 acts as a good balance in terms of collision and planning success.

has fundamentally more solutions and more likely to yield robust results. Some qualitative planning results are shown in Fig. 3.8.

3.4.6 Ablation Study

We conduct ablation study on the influences of two important design choices in our planner: the discretization n defined in Sec. 3.3.2 and the smoothness constant λ defined in Eq. (3.2). The results are presented in Fig. 3.11. By increasing n , a fine-grained trajectory better approximates a continuous curve and therefore induces better smoothness and planning success. The smoothness constant λ can be treated as a kinematics constraint relaxation and combined with η_{motion} to model the size of the trust region ellipsoid.

3.5 Conclusion

We have presented a joint motion and grasp planning approach based on first-order trajectory optimization to generate grasping trajectories. Similar to other optimization-based planning methods, our approach is by nature local, and it has no guarantee to find feasible solutions. However, compared to sequential approaches, where a grasp is chosen from a set of pre-computed grasps, our method considers the whole set of feasible goal configurations in an online scheme, which is more efficient and principled. Our OMG Planner does not require a perfect goal configuration, since the

candidate goals are determined during the planning process. A goal set is explicitly modeled as a probability distribution and the best goal is selected online. Moreover, our method synthesizes new grasps online to augment the goal set. Our experiments demonstrate that the OMG Planner can plan grasping trajectories for cluttered scenes efficiently and robustly. While our work is based on CHOMP, we believe that the online goal selection and refinement procedure can also benefit other planning methods and tasks that involve a set of goals.

In planning, we assume access to full object model and environment. This is useful if we have a nearly perfect perception module and interact with known objects in a fixed environment. On the other hand, large-scale model-free RL methods [88, 68, 54] mostly focus on top down grasping. How to map perception to more complicated 6 Dof control remains a challenge. While long-horizon tasks with sparse rewards require large amount of environment interactions for RL methods, we would like to investigate if supervision can help the sample efficiency in end to end vision-based robot manipulation. Moreover, to achieve minimal transfer from simulators to real-world, we want that learner to have an awareness of the workspace geometry by treating points as input. Thus, we have reached to the final control stage of an end-to-end system that aims to coherently integrate the perception and planning modules.

3.6 Note

The ideas and figures within this chapter have appeared previously in the peer-reviewed publication [129].

Chapter 4

IMITATION AND REINFORCEMENT LEARNING FOR GOAL CONDITIONED GRASPING POLICY

4.1 Introduction

For many robotics tasks with the presence of demonstrations from either humans or designed controllers, imitation learning [87] (IL) proposes to train control policies via standard supervised learning methods. This simple and straightforward training approach is dubbed behavior clone. By maximizing the likelihood of actions provided by an expert demonstrator, imitation learning can produce effective policies. Moreover, as deep learning have pushed supervised learning algorithms to the point of being robust and reliable, imitation learning algorithms have demonstrated success for generating control policies from high-dimensional sensory data such as images [89]. However, with a batch data of limited diversity, imitation learning can suffer from the cascaded error issue [102]. An popular alternative paradigm to imitation learning is reinforcement learning (RL), which holds the promise to learn from trials and errors. Despite years of research, many RL methods still lack convergence guarantees [123] and require prohibitive numbers of samples [107]. In practice, many commonly used algorithms are extremely sensitive to hyperparameters [47]. These problems are even more severe with the presence of function approximators.

An interesting direction is therefore to combine the best of these two worlds: use RL to minimize covariates shifts by online rollouts and even surpass experts, and use IL for stabilizing algorithm and improving sample efficiency. In this work, we are interested in position control in operational space with the observations taken from end effector's view. The goal is to grasp a single target object with random shape. We use the previous planning work [129] as supervisions for grasping in the cluttered scene. We experiment with mixing supervised learning methods Behavior Clone (BC) and Monte Carlo Return Estimate (MCRE) with reinforcement learning methods deep deterministic policy gradient (DDPG).

Learning to reach a goal state [51] can circumvent the problem of sparse rewards and achieve

useful generalization across tasks. The key observation is that in the setting where the tasks correspond to reaching different goal states, every trajectory is a successful demonstration for the state that it actually reaches, even if it performs sub-optimally for the goal that was originally commanded [37]. Therefore, the rollouts from policy itself or experts can be used to supervise a goal conditioned policy. In robot manipulation, the goal state can be a final grasp configuration or intermediate way-point, and a policy that learns to reach any configuration can be extremely useful. We make use of 6DGraspnet [80] that uses conditional variational autoencoder for generating feasible grasps. We use the supervision data to finetune the sampler and evaluator for goal selection and refinement for our grasping policy. Shown in Fig. 4.1, our goal-conditioned framework learns to generate, select, refine, and reach goal. In addition to goal conditioned supervised learning, in our egomotion predic-

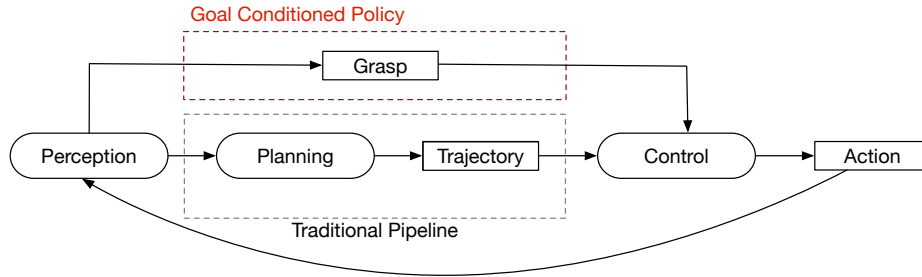


Figure 4.1: Different from traditional sense, plan, act paradigm, an end-to-end goal conditioned policy for control uses goal as a proxy to circumvent long-horizon planning.

tion, we address the cascaded error in behavior clone with a trajectory supervision loss. In particular, we unroll the policy in training and compute the state matching loss with the conditioned goal state. While it does not address the covariate shift problems with extra online rollout, it enforces the final state to be close to the expert state. This strategy has similarity to n -step return in value function learning, which enforces some regularization from rollouts.

Lastly, we explore sample-efficient off-policy RL algorithms that can leverage a pretrained behavior clone policy to improve grasping success from environment interactions. Similar to [126], we plan to build on top of deep deterministic policy gradient to circumvent the need for reward shaping. Moreover, by incorporating demonstration data, these approaches are often found more robust to environment noises and have more human-like behavior. Lastly, these approaches often can

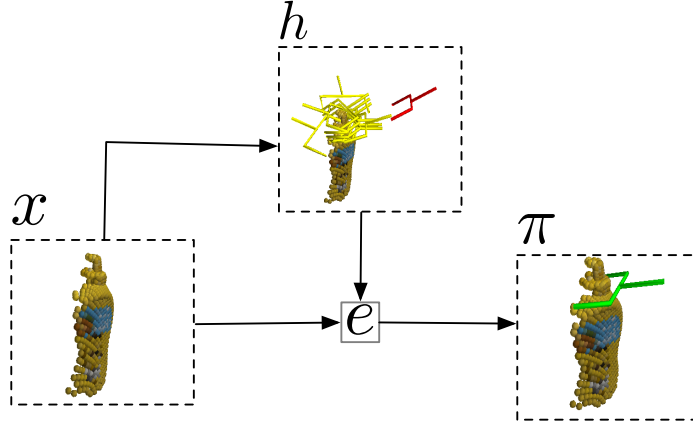


Figure 4.2: Our Goal Conditioned Grasping policy framework. Our grasp framework (sampler h and evaluator e), based on [80], takes in the observed point cloud x and synthesize a goal grasp for the policy π . The red gripper on the top denotes current gripper pose, yellow denotes sample grasps, and green denotes the selected grasp after refinement.

reduce the required samples by initializing the policy with imitation learning. Our entire framework Goal Conditioned Grasping (GCG) policy, shown in Fig. 4.2, makes the following contributions: (1) An end-to-end visuomotor learning framework that combines imitation and reinforcement learning. (2) A learned goal selection and refinement mechanism to enable goal conditioned policy. (3) A robust goal-reaching policy that outperform several baselines in grasping diverse objects in simulation domain.

4.2 Preliminaries and Related Works

4.2.1 Markov Decision Process

Formally, We model the control problem as a Markov decision process (MDP), which is defined using the tuple: $\mathcal{M} = \{\mathcal{S}, \mathcal{R}, \mathcal{A}, \mathcal{T}, \rho_0, \gamma\}$. $\mathcal{S}$ and $\mathcal{A}$ represent the state and action space. $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function. For most top-down grasping tasks, the reward is an indicator function given at the end of the episode denoting if the target object is lifted and there's no collision. $\mathcal{T} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ is the transition dynamics. In model-free RL, we are only allowed sample access to this function. ρ_0 is the probability distribution over initial states and $\gamma = [0, 1)$ is a discount factor. Our goal is to solve for a policy of the form $\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}_+$ which optimizes the expected returns:

$$\eta(\pi) = \mathbb{E}_{\pi, \mathcal{M}} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]. \quad (4.1)$$

We also define value, Q, and advantage functions associated with a policy as follows:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\pi, \mathcal{M}} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \\ Q^\pi(s, a) &= \mathbb{E}_{\mathcal{M}} [R(s, a)] + \gamma \mathbb{E}_{s' \sim \mathcal{T}(s, a)} [V^\pi(s')] \\ A^\pi(s, a) &= Q^\pi(s, a) - V^\pi(s) \end{aligned}$$

4.2.2 Model-based Approach

Model-based control methods assume some knowledge about the dynamics (model). In grid-based setting with known stochastic transition model, value iteration and policy iteration [116] has closed-form update and good convergence guarantees. For continuous control, stochastic optimal control algorithms such as iterative Linear Gaussian Policy (iLQG) [120], Path integral for Policy Improvement (PI2) [119] and Model Predictive Path integral (MPPI) [131] or Model Predictive Control (MPC) approaches such as [101, 41] have demonstrated various kinds of success in robotics. Specifically for modelling the contacts, [78, 122, 91] have shown impressive trajectory optimization in simulators and real world. More recently, model-based methods applies function approximation such as a neural net to model the dynamics from the collected data. By first learning the dynamics of the system in a supervised learning way, it can exploit off-policy data. Then, model-based methods use the model to derive controllers. Guided Policy Search (GPS) [67, 77] splits the problem into a “control (C) step” that finds multiple simple local policies for different initial states and a “supervised (S) step” that optimizes the global policy. [27] uses a deep net to generate visual foresight for manipulation tasks. Most deep model-based RL works use the learned model as a black-box simulator to generate samples [61]. Iterative Learning Control [9] uses the actual rollouts to remove the bias of model rollouts. [71] combine MPC planning online with model and terminal value learning offline. [130] benchmarks model-based controllers in simulation domains. More recently [97, 18] attempt to combine policy learning with model learning. While it’s not trivial to acquire an exact model of the real world dynamics, these methods are orders of magnitude more efficient in sample data.

4.2.3 Model-Free Value-based Methods

When the state space is huge or the transition model is unavailable as in most robotics setting, Approximate dynamic programming or neuro-dynamic programming [116, 6] apply function approximators such as neural net to fit a value function or policy. In such cases, fitted value or policy iteration with function approximation have no guarantee to converge and can even diverge. On-policy methods such as SARSA and TD-Learning [116] use online rollouts to fit a state or action value function. These methods are not sample efficient as other on-policy methods.

Off-policy methods can make use of past data or data that come from other sources. RL methods that can work with off-policy data: soft actor critic (SAC [43]), Double Deep Q Network (DDQN [125]) and Monte Carlo Return Estimate (MCRE [116]). DQN [76] aims to estimate the Q-function by minimizing the Bellman error, given by

$$\mathcal{E} = \frac{1}{2} \mathbb{E}_{s,a} [(Q_\theta(s, a) - (r(s, a) + \gamma \max_{a'} Q_\theta(s', a'))^2] \quad (4.2)$$

Observe that the target for estimation is $\max(\mathbb{E}[Q_\theta(s', a')])$, which is not greater than the regression $\mathbb{E}[\max(Q_\theta(s', a'))]$ tautologically. To reduce the effects of overestimation in the next states introduced by bootstrapping, DDQN introduces a target network to decouple the correlation.

Monte Carlo Value Estimate simply tries to minimize the loss for Monte Carlo policy evaluation:

$$\mathcal{E} = \frac{1}{2} \mathbb{E}_{\tau \sim \pi} [\sum_t (Q_\theta(s_t, a_t) - (\sum_{t'=t}^{\infty} \gamma^{t'-t} r(s_{t'}, a_{t'}))^2] \quad (4.3)$$

This resembles supervised learning and requires no bootstrapping. However, it's only unbiased when we use on-policy data. Compared with model-based methods that has bias in modeling the world, model-free methods often enjoy better asymptotic performance and suffer worse sampling efficiency. These value estimation method with function approximation are used by many top-down grasping frameworks [54, 68, 111].

4.2.4 Policy Search

Policy search methods optimize a parameterized policy $\pi_\theta(u_t|x_t)$ over actions u_t conditioned on the state x_t . The goal is to minimize expected trajectory cost under the policy $J(\theta) = \sum_{t=1}^T \mathbb{E}_{\pi_\theta(x_t, u_t)} [J(c(x_t, u_t))]$ given cost $c(x_t, u_t)$ and dynamics $p(x_{t+1}|x_t, u_t)$. Denote the marginal of a trajectory $\tau = \{x_1, u_1, \dots, x_T\}$

as $\pi_\theta(\tau) = p(x_1) \prod_{t=1}^T p(x_{t+1}|x_t, u_t) \pi_\theta(u_t|x_t)$. The naive REINFORCE algorithm uses samples to compute

$$\nabla_\theta J = \mathbb{E}_\tau \left[\left(\sum_{t=0}^T \nabla_\theta \log \pi_\theta(u_t^{(i)}|x_t) \right) R(\tau^i) \right]$$

[52] proposes a natural policy gradient method (NPG) to represent steepest descent direction based on the underlying structure of the parameter space. [4] further develops policy search that is covariant in the trajectory probability space.

[96] benchmark the performance of different continuous control algorithms including Trust-Region Policy Optimization (TRPO) [107] and NPG [52] with linear approximation and RBF kernel and a focus on diverse trajectories. However, even with state information, these approaches still require millions of interactions to learn a single task and exhibit large variance to hyper-parameter and reward design. Compared with the value bias introduced by value-based methods, these methods have an even worse sampling efficiency but better training stability because they are optimized directly for the RL objective.

4.2.5 Actor-Critic Methods

The combination of value and policy methods, named actor critic algorithms [58], strives to combine the best of two worlds. Policy gradient theorem connects policy gradient with value estimation.

$$\nabla_\theta J = \mathbb{E}_\tau \left[(\nabla_\theta \log \pi_\theta(u_t^{(i)}|x_t)) A_\phi^{\pi_\theta}(x_i, u_i) \right]$$

In theory, the empirical mean estimates the expectation over static distribution of the policy on the MDP. There are standard off-policy algorithms such as Q-learning to train the quality function. Alternatively, a value function can be trained with sample estimate. The advantage is used to reduce the variance of policy improvement. Algorithms such as TRPO and DDPG in practice also adopt a baseline as actor critic method. In continuous control settings, off-policy methods often show better data efficiency and require less on-policy exploration. [90] builds on Deep Deterministic Policy Gradient (DDPG) to learn general and robust control policies. By using a deterministic policy gradient [109] jointly with a Q function, the gradient becomes straightforward to compute and often requires less samples

$$\nabla_\theta \hat{J} = \mathbb{E}_\tau [\nabla_a Q(s, a) \nabla_\theta \pi_\theta(s)] \quad (4.4)$$

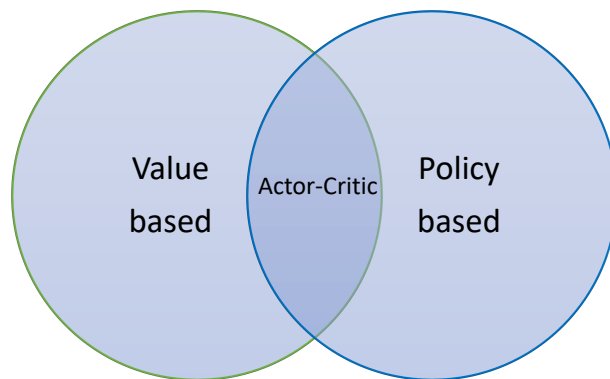


Figure 4.3: Actor-Critic Method train value and policy functions simultaneously.

Soft Actor Critic (SAC) [43] adopts max entropy reinforcement learning, whose objective is to maximize the entropy-regularized sum of rewards. The entropy regularization helps the policy to explore the state space and found more robust to noises and tasks with multimodality.

4.2.6 Imitation Learning with Generalization

A naive and widely used way to learn from an expert policy is to do behavior clone (BC), which reduces to supervised learning problem. The line between inference and sequential decision making has to be made clear. First, predictions made by inference algorithms are explicitly assumed to have no effect on the world in which they operate. Second, Robotics system often make use of sophisticated planning method on a defined cost to achieve long-term performance. Ignoring these planning algorithms in lieu of a reactive learning approach often leads to poor, myopic performance.

The first point can be due to overfitting: perhaps our training data set is too small to produce a good generalization performance. Due to the cascaded error issues and the covariate shift [102], an error of $O(\epsilon T^2)$ is inevitable for a learner with average ϵ step error in horizon T . A solution to this problem relies on interaction: interleaving execution and learning. In particular, at each iteration of the algorithm, the current learned policy is executed and new dataset is collected and augmented [102].

On the other hand, Inverse Optimal Control (IOC) strives to infer the expert's cost function through demonstrations. IOC methods rely on the observation that cost functions generalize more

broadly [84]. The approach of [135] tries to explain the expert’s behavior with the maximum entropy model. In practice, inverse RL methods often requires solving RL in the inner loop which can be expensive. Recently [49] use generative adversarial model to learn a discriminator, [34] establishes guided cost learning for policy optimization, [33] make a connection between generative adversarial networks, inverse reinforcement learning, and energy-based models.

To learn the RL tasks with imitation learning, [95, 134, 126] use demonstration data to circumvent the need for reward shaping and guide exploration for policy gradient methods. [114, 113] leverage expert oracle in training time to reduce sample complexity than plain RL for control tasks.

4.2.7 Goal-Conditioned Policy Learning

In cases of sparse rewards in long horizon or multi-task learning, pure RL agents suffers heavily from the difficulty of getting any rewards with exploration. While reward shaping requires heavy engineering and imitation learning requires expert demonstrations, using specified goal to represent reward might simplify the problem [51]. A policy that can reach any specified goal in general can be extremely useful for robot learning. For instance, one can decompose a long horizon tasks into sub-horizon with subgoal synthesized by a neural network. Moreover, goal in other form such as image [110] can be provided to train RL learning algorithm, which can be more scalable for many human tasks. For vision-based 6-dof grasping, grasp database [40] or grasp synthesis methods [80] can be used to specify goal. [2, 23, 37] make the observation that every trajectory is a successful demonstration for the goal state that it actually reaches, thereby relabeling goals in rollout trajectories for effective learning.

4.2.8 End to End Vision-based Robot Manipulation

Many learning-based methods perform robot manipulation directly from high dimensional input such as RGB images. [66] propose to use Guided Policy Search (GPS) for end-to-end visuomotor control. [93] benchmark various off-policy methods for top down grasping in simulators. [88, 68] makes use of large-scale self-supervision data to learn vision-based grasping. [79] deploy closed loop grasping pipelines based on grasp detection algorithms. While a static RGB sensor is often hard to reason about the 3D complexity of the manipulation tasks and found poor generalization,

depth point clouds and voxels often require nontrivial registration and calibrations. Some recent works [14, 42] use active learning to determine successive sensor placements to improve state estimations. [1] use multiple sensors to integrate more information in states to synthesize controls. More recently, [111] use fusion as model to generate action view, and efficiently train a Q network for closed-loop grasping. Overall, end-to-end control systems have achieved some impressive results on manipulation tasks. While these works are mainly for top-down grasping in simulation domains, in real worlds, for instance a kitchen robot, It’s desirable to have 6DOF grasping and collision avoidance in cluttered scenes. The main challenges different from top-down grasping is that the object can fall down with an inappropriate touch, and the camera might easily lose the views during exploration. Different from grasp synthesis works that focus solely on final grasp pose and end-to-end grasping that lacks structure and interpretations, we propose to jointly sample, select, refine, and reach goal in a single framework.

4.3 Methodology

We formulate the task of end-to-end 6 Dof grasping as a goal-conditioned policy learning problem where the goal is synthesized and selected on the fly. Our goal is to learn an ego-centric position-control policy π for end effector. We represent the observation s as the partial view of point cloud of the object to be grasped. We use an egocentric point cloud observation for better generalization and learning performance. The action a encodes the relative rotation and translation between the current end effector pose and the next target pose. The reward r is 1 if the target object is lifted at the end of the episode and 0 otherwise. Each episode has a maximum horizon T . The goal state g is a relative pose from the current state’s observation.

We collect demonstration data $\mathcal{D} = \{s_0^j, a_0^j, s_1^j, \dots\}_{j=0}^D$ and generate state-action-goal tuples (s_t^j, a_t^j, g_t^j) . There are two regression tasks that we can train on with these supervision data: (1) goal conditioned behavior clone (GCBC) that regresses action from state and goal. (2) goal supervision (GS) that regresses goal from state.

$$L_{\text{GCBC}} = \mathbb{E}_{(s_t^j, a_t^j, g_t^j) \sim \mathcal{D}} \left[\left\| \pi_{\theta}(s_t^j, g_t^j) - a_t^j \right\|^2 \right] \quad (4.5)$$

$$L_{\text{GS}} = \mathbb{E}_{(s_t^j, g_t^j) \sim \mathcal{D}} \left[\left\| f_{\phi}(s_t^j) - g_t^j \right\|^2 \right] \quad (4.6)$$

where π_{θ} parametrizes our policy and f_{ϕ} parametrizes our goal synthesizer.

4.3.1 Grasp Sampling, Selection, and Refinement

Similar to [83], we want to generate goals with a generative model for our goal-conditioned policy. We briefly revisit the grasp sampler and evaluator proposed in [80] and refer the interested readers to the original work. We use the implementation [72] which uses a new sampler framework trained with Implicit Maximum Likelihood Estimation (IMLE) [69]. Assume the latent vector lies in a unit grid of dimension d . Denote a grasp database of object as $\mathcal{G}$ of N grasps and grasp sampler parametrized as h . Given a point cloud s and m noise vectors $z_{0,\dots,m} \sim [0, 1]^d$, the sampler generates grasps $\hat{\mathcal{G}} = \{h(s, z_i), 0 \leq i \leq m\}$ and minimizes a reconstruction loss

$$L_{\text{sampler}} = \frac{1}{N} \sum_{g^* \in \mathcal{G}} \min_{\hat{g} \in \hat{\mathcal{G}}} \|\mathcal{T}(g^*; x_h) - \mathcal{T}(\hat{g}; x_h)\|_1 \quad (4.7)$$

where x_h is a set of predefined points on the robot gripper. Under appropriate condition, IMLE can be shown to be equivalent to maximum likelihood which is the opposite of Generative Adversarial Network (GAN), and therefore prohibits problems such as mode collapses. The evaluator remains the same as the original work, which takes a masked point clouds of object and gripper points, and predicts a score denoting the probability of success applying this grasp. It's trained with supervised learning separately.

For a grasping policy, it's desirable to have diverse sampled grasps due to the multimodality of the task. On the other hand, the data-driven evaluator should preserve a grasp selection and refinement procedure. Note that our expert planner [129] uses online learning to favor goals that are "optimal" in terms of its optimization objective. We therefore uses the final grasp in demonstration to finetune the sampler and evaluator. Specifically for state s and grasp g^* , our sampler generates a set of n candidate grasps $\mathcal{G}_n = \{\hat{g}_0, \hat{g}_1, \dots, \hat{g}_n\}$. The target for evaluator, denoted by e is adversarial: it needs to classify all samples in $\mathcal{G}_n$ as 0 and groundtruth grasp g^* as 1.

$$L_{\text{evaluator}} = -(\log(e(g^*)) + \sum_{\hat{g} \in \mathcal{G}_n} \log(1 - e(\hat{g}))) \quad (4.8)$$

Effectively we hope to learn an evaluation function that can select grasp for our policy given a set of candidates. To refine the sampling region in latent space, one could apply Cross Entropy Method (CEM) or Upper Confidence Bound (UCB). To directly refine the selected grasp, one can use gradient descent or Metropolis-Hastings sampling.

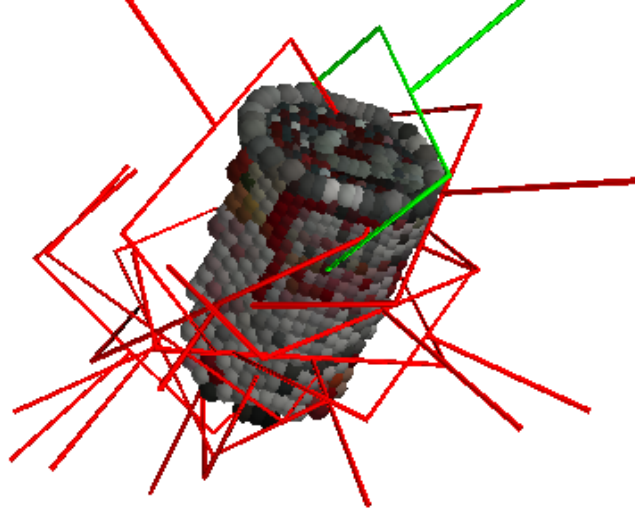


Figure 4.5: Green denotes the grasp with highest value and red denotes other sampled candidate grasps, which shows that the finetuned value function remains picking high quality grasp.

4.3.2 Learning to Reach Goal

Given a goal sampling and selection mechanism, we have a goal mapping $\mathcal{S} \mapsto \mathcal{G}$. Then, goal conditioned behavior clone in Eq. 4.6 can be used to supervise our policy π_θ . Following [2, 23, 37], we can add horizon information to our state and relabel goals for more effective learning. The key observation is that in the setting where the tasks correspond to different goals, every trajectory is a successful demonstration for the state that it actually reaches. We will say a policy is optimal if it maximizes the probability the specified goal is reached

$$J(\pi) = \mathbb{E}_{g \sim p(g)} [P_\pi(s_T = g)] \quad (4.9)$$

This allows online rollouts of the learned policy and usage of subgoals. An arbitrary trajectory from a sub-optimal policy may be suboptimal for reaching the intended goal, it may be optimal for reaching some other goal. Specifically while the label stays as a_t , our policy now commands $\pi(s_t | s_{t+h}, h)$ where $0 < h < T - t$. Overall we can relabel every such combination to create $\binom{T}{2}$ optimal tuples from a single trajectory, which squares the number of data.

Note the notion of optimality does not have to take the direct or shortest path to the goal, it simply has to eventually reach the goal. Therefore, we propose to explicitly supervise the rollout reach state $\hat{s}_{t+h}$ with s_h . One might be tempted to also supervise intermediate states or actions but

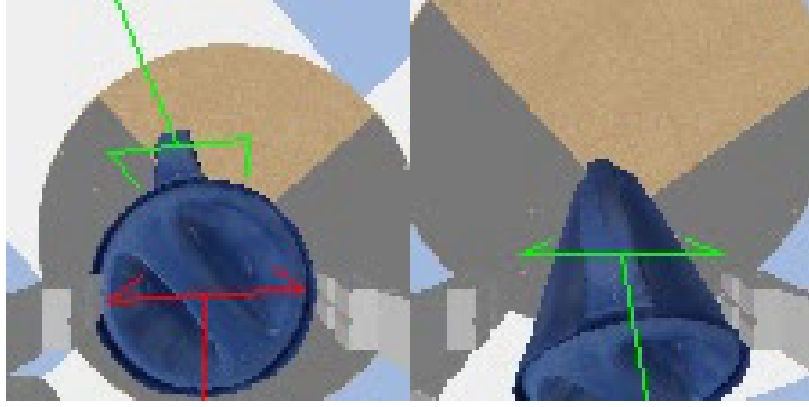


Figure 4.6: The figures show two grasping episodes to compare reaching good grasp to reaching a bad grasp. Red denotes a bad sampled goal grasp and green denotes the grasp selected by planner [129].

these are not necessarily optimal. Directly supervising only the state reached with the subgoal can result in unstable rollout that takes nonsmooth motion (for instance, the policy can reach subgoal in one step and remains still). Therefore, we propose our trajectory centric behavior clone (TCBC) loss

$$L_{TCBC} = \mathbb{E}_{(s_t, a_t, s_{t+h}, h) \sim \mathcal{D}} \left[\|\hat{s}_{t+h} - s_{t+h}\|^2 + \sum_{k=1}^h \|\Delta a_{t+k}\|^2 \right] \quad (4.10)$$

$$\hat{s}_{t+k+1} = \mathcal{T}(\pi_{\theta}(\hat{s}_{t+k}, g_{t+k}, h - k)^{-1}; \hat{s}_{t+k}) \quad (4.11)$$

$$\Delta a_{t+k+1} = a_{t+k+1} - a_{t+k} \quad (4.12)$$

In practice, we include the information of goal and current state using masked point cloud as in [80]. Note the current hand pose always stays at origin since our policy is egocentric. Our input point cloud is then $\mathcal{T}(g_t; x_h) \cup s_t$ and a binary class mask. For the same reason, our supervision for goal state s_{t+h} would come from inverting the action transform and applying it to the object point cloud. To fully explore the performance of goal-reaching policy, we can combine subgoal generation in training, similar to [73, 30].

4.3.3 Augmenting Imitation Learning with On-Policy Rollouts

In this section, we attempt to improve the algorithm performance with additional environments interactions, similar to [23]. There are a few observations we make on the failure cases of the pretrained policy. Complex contact dynamics is not easy to learn through imitation, and it's also not explicitly modeled in the planner. The policy can therefore struggle with contacting large objects such as box and often fail to recover when it gets stuck. Through online rollouts from an already well performed policy, we expect the network to improve on those failure cases. We adopt two frameworks [126, 95] to incorporate expert data and pretrained policy. [126] use off-policy Reinforcement Learning algorithm DDPG with demonstration in replay buffer. Specifically we fit the quality function $Q(s, a, g)$ and then compute the deterministic policy gradient as

$$\nabla_{\theta} \hat{J} = \mathbb{E}_{(s,a,g) \sim \mathcal{D}} [\nabla_a Q(s, a, g) \nabla_{\theta} \pi_{\theta}(s, g)] \quad (4.13)$$

Although we use simulators in our training procedure, we want to avoid requiring a size of millions of environment interaction steps. This approach makes good usage of expert data for exploration and replay buffer. Note that a prioritized replay buffer and a combination of 1-step return with n-step returns are used. Similarly, we initialize the policy with imitation learning and initialize Q function with supervised training from Monte Carlo return estimate. [2, 82] have applied similar ideas to manipulation setting.

[95] uses on-policy rollouts and stochastic policy gradients to directly finetune the imitator. To capture all information present in the demonstration data, they add an additional term to the gradient update:

$$\nabla_{\theta} \hat{J} = \sum_{(s,a,g) \sim \mathcal{D}_{\pi}} \nabla_a A(s, a, g) \nabla_{\theta} \log \pi_{\theta}(a|s, g) + \sum_{(s,a,g) \sim \mathcal{D}} \nabla_a w(s, a, g) \nabla_{\theta} \log \pi_{\theta}(a|s, g) \quad (4.14)$$

where $\mathcal{D}_{\pi}$ denotes the on-policy rollouts and $w(s, a, g)$ is a heuristic weighting scheme to balance imitation and reinforcement learning.

4.4 Experiments

In this section, we first discuss implementation details of our framework. We then strive to answer the following questions: (1) Can end-to-end goal-conditioned policy learning perform better than

behavior clone? (2). How much improvement can RL bring to a pretrained model? We also conduct ablation study on a few design choices made in our method.

4.4.1 Implementation

Environment

We conduct experiments with the Panda Franka arm, a 7-DOF arm with a parallel gripper. The scenes are generated by dropping sampled YCB objects [11] with random upright poses on a table in the Pybullet Simulator [20]. Each scene samples a target object. The objects used are cracker box, sugar box, tomato soup can, mustard bottle, potted meat can, and bleach cleanser. The expert trajectories length are controlled to be not greater than 20 timesteps and the maximum horizon $T = 30$. To not completely lose previous observed points in history, we accumulated observed point clouds in an exponential moving window fashion. Our observations are sampled 1024 points from these accumulate points.

Training

The feature extractors for all networks use PointNet++ [92]. Policy and value functions are all represented as three layer fully connected network. We use Adam as our optimizer and decrease learning rate in a piece-wise step schedule. We randomize initial states from a set of pre-generated scenes. We use clipped-PPO [108] as the policy gradient method for DAPG. We do rollouts of expert data online and use a replay buffer of size 5000. Similar to [54], since the reward is bounded by 1, we use cross entropy as the training signal for Q and value function training. The goal generation and selection is trained separately from goal-reaching policy, which means that the goal-reaching policy always use the goal selected by expert during training.

Testing

For the goal reaching component, one can generate grasps for local frame at each environment step. However, we note that the grasps in the world coordinate is valid. To keep the best grasps, we greedily keep the best grasp up to the current step as the selected grasp. Moreover, to efficiently use goal synthesizer, we generate grasp candidates every 3 frames. The testing scenes and initial

20 scenes per object, 5 runs			
Object Name	BC	GCG	Traj-BC
Cracker Box	0.56	0.83	0.86
Sugar Box	0.91	0.96	0.96
Tomato Soup Can	0.55	0.77	0.72
Mustard Bottle	0.71	0.73	0.75
Bleach Cleanser	0.83	0.92	0.88
Mean	0.712	0.842	0.834

Table 4.1: Comparison for our imitation learning algorithms. Our goal conditioned framework performs more stably for all objects. The addition of trajectory state loss has a comparable performance.

states are loaded from pre-generated holdout dataset solved by expert to make sure the problem has at least one solution.

4.4.2 Goal-Reaching Imitation Learning

We evaluate the performance of our goal reaching policy, shown on Fig. 4.7. Our results in Table 4.1 show that our method has a large improvement over the baseline without goal information, similar to the observation of [2, 82]. For difficult object such as mustard bottle, the grasps cluster around the head and two sides, while other grasps can easily result in slipping finger closure. We therefore conclude that encoding the goal gripper point cloud to reach along with observed point cloud guides the policy toward pregrasp configuration. Moreover, we observe a comparable performance through our trajectory behavior clone loss which penalizes state deviation. As addressed above, the trajectory loss would more appropriately work with subgoal policy, which is an interesting future direction. Since the grasping component is trained separately, we use the same grasping for a fair comparison.

4.4.3 Finetuning with RL

In this section, we finetune the Goal-conditioned policy with reinforcement learning. For finetuning using DDPG, we use a mix of expert rollouts and on-policy rollouts at training, and gradually shift from behavior clone to deterministic policy gradient. We use a pretrained network from Behavior



Figure 4.7: The figures show examples of successful grasping episodes from our learned policy in the simulator [20].

20 scenes per object, 5 runs		
Object Name	GCG	GCG-Finetune
Cracker Box	0.97	0.95
Sugar Box	0.99	1.00
Tomato Soup Can	0.79	0.91
Mustard Bottle	0.65	0.78
Bleach Cleanser	0.92	0.94
Mean	0.864	0.916

Table 4.2: We note that the experiments on this table use goals from the expert, and therefore the performance is slightly better than Table 4.1. The table and empirical observation show that RL improves the performance over difficult scenes.

Clone and asymptotically mix behavior clone loss with policy loss determined by the critic. This results in a smooth transform from imitation to reinforcement learning. The critic target is a mix of 1 step Bellman target and the n -step episode return. For the experiment, we use the goal selected by planner as input for the goal-conditioned policy for a fair comparison. Based on the table 3.3, the preliminary results indeed show that allowing the robots to interact with environment has a positive impact on the policy. We empirically also observe that deterministic policy often behave more compatible to use with imitation learning.

4.5 Conclusion

Overall, we present an visuomotor framework that naturally leverages goal information for 6Dof grasping. Specifically, our framework builds on top of previous work to provide demonstration data

on simulators. We then train a grasp sampling and selection mechanism to specify goal. We also apply goal conditioned supervised learning with trajectory-based behavior clone to learn the policy. Finally, we augment the algorithm with online rollouts and reinforcement learning. Our preliminary experiments demonstrate the performance of our learned policy. Our preliminary imitation learning experiments demonstrate that goal information improves the robustness and precision of a control policy. Our preliminary reinforcement learning experiments show that model-free RL algorithm has a potential to finetune policy by online rollouts and conquer difficulty for supervised learning such as contact dynamics and failure recovering. Since our method adopts point cloud observation and an efficient number of environment interactions, we believe its transfer in real world environment is promising. Immediate next steps include experimenting with larger sets of objects, investigating more explicit goal-reaching policy with subgoal generation, and integrate goal-conditioned RL algorithms in our problem setting in a more principled way.

4.6 Note

The work in this chapter is still an ongoing research, so the content is amenable to changes in the future version.

Chapter 5

AFTERWORD

This thesis studied how a robot system can make use of goal information to improve performance in perception, planing, and control. Chapter 2 demonstrates that a neural network can be used to iteratively refine object pose estimate to match against observed image. This network is trained in a standard machine learning setting with a batch dataset. We further develop tracking system and other applications of the framework. Chapter 3 shows that for manipulation planning problem, a trajectory optimization method can be improved with online goal selection and refinement. The goal selection follows an online learning setting, where the optimization objectives become the data observation in sequence and the algorithm aims to minimize regret compared with best goal in hindsight. Finally, Chapter 4 describes our approach to learn a 6-dof goal-conditioned grasping policy. We attempt to combine imitation and reinforcement learning to use offline data and learn efficiently online. We examined our methods on a table top manipulation setting, and our experiments indicate the advantages and drawbacks compared with related works or baselines. Admittedly, many evaluation experiments should be done in a large scale across many platforms. The intuition for this research is that when human attempts to manipulate an object, one has a rough idea of what to estimate or act, and therefore use those implicit information in a feedback loop. Similarly, claimed as universal function approximators, neural nets are now widely used to learn the entire system. Different learning setting and algorithms adopted in these “end-to-end” fashion should thus also use goals as constraints or extra information.

As an undergraduate researcher, I have attempted to explore the broad spectrum of robotics theory and application. While this certainly has led to only shallow investigation on specific idea, luckily it also opens many high-level research directions for future work:

5.0.1 *Improved Learning Methods for Robotics*

The increase of computes for modern GPUs is an important reason for the raise of deep learning in inference problems in recent years. Machine learning, as an application of statistics, holds the promise to learn from data. Different from computer vision and natural language processing, acquiring data is not entirely free in robotics and they often do not have i.i.d assumptions. There are two kinds of data that one can collect, corresponding to two mainstream online decision-making algorithms. The first kind generates data from humans or existing systems through sensor recordings. The amount of these sorts of data can be abundant and can be used by imitation learning algorithms. While imitation learning algorithms have developed into its own kind and separated from supervised learning, they still tend to focus either on the problem of compounding errors or the need for learning deliberative strategies [3]. An interesting direction is to develop algorithms to address both issues simultaneously. While many works propose to learn from state action pairs, experts like planners often generate the whole trajectory to solve a task. While IOC is a feasible solution to learn from demonstration trajectory, it would also be interesting to see if there are other formulations.

In some scenarios, collecting such data can be expensive and is often not automatic. The ideal way is to allow the robots to learn from interacting with the world. The famous Reinforcement Learning algorithms comes to model it as a stochastic optimal control problem in a MDP. While many proposed algorithms has long history but many problems still exist in both theory and application of these algorithms. With a deep neural network parametrized with potentially millions of parameters, a generalized algorithm such as policy gradients needs to explore the state action space with large-variance batch samples. This can result in very inefficient and bad convergence. For relatively expensive robotic interactions that need to be deployed in the real world, requiring large samples can be unrealistic and dangerous. Moreover, this hasn't considered the multi-tasking, generalization, and robustness of the learned policy. Many interesting research directions such as model-based RL, purely offline RL, meta learning, goal-conditioned RL, constrained MDPs are all potential ways to reduce the required environment interactions and improve the explorations. Another issue with the current RL method is that they are often too sensitive to various kinds of factors and the benchmark performance are not easy to reproduce. An immediate step would be to improve

benchmark and reproduction of these methods. I would also suspect new formulations of robot learning might be proposed in the future research.

5.0.2 *Improved Robotic System*

It's fundamental for robotics research to be applied in the real world settings. Specifically for manipulation system, most planning and control algorithms currently adopted in the real world don't contain a learning component in it. Compared with the most common "sense, plan, and act" pipeline that might seem contrary to human behavior, robots with learning has limited generalization, interpretations, and robustness to function well in human-centric environments. An ideal strategy to build end-to-end system is to based on top of the modularized components, in a composed and hierarchical fashion, and then have a universal way to convert to an end-to-end framework. After all, many domain knowledge and explicit bias from traditional methods can be used to guide learning. Assuming that there can be unavoidable noises in perception and reactive motion, the integrated manipulation systems that map from observation to action can then adapt and be trained further. Taking a biased example for instance, assume that we can convert most robotics tasks to the standard goal reaching task. Consider that the perception has pose error α , planning generates goal β worse than the optimal goal, and control has γ error in reaching the goal. Then the question is to find a principled way to integrate these system and a learning algorithm to train it end to end such that the learned loss is better than the naive cascaded error.

Benchmarks such as [133] propose many manipulation tasks in a simplified setting. An obvious direction is to continue the works on solving robot tasks in simulators and hope that simulators, specifically simulated observation and dynamics simulation, can be made with high fidelity. One interesting direction that has close relationships to all machine learning algorithms is that if one can generate interesting noise distributions to improve the generalization. For instance in self-driving cars, it's impractical to expect the drivers or systems themselves to explore the edge cases enough frequently. If that's the case, then empirical risk minimization in machine learning would mainly minimize the loss on the training distributions, which can prohibit the ability to handle emergency cases. For robotics perception, it appears that generalization across observations and state estimations can be separate topics in fields such as computer vision. However, when the

problem is well defined, these approaches can also have the potentials to be applied in the real world. For robotics control, low-level policy can be harder to train but allow more adaptations. A behavior to output motor torques to reach any specific goal can be extremely difficult to learn but is widely applicable for many robotics tasks. On the contrary a control policy without 6 degrees of freedom can be hard to work in a general manipulation setting.

BIBLIOGRAPHY

- [1] Iretiayo Akinola, Jacob Varley, and Dmitry Kalashnikov. Learning precise 3d manipulation from multiple uncalibrated cameras. *arXiv preprint arXiv:2002.09107*, 2020.
- [2] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in neural information processing systems*, pages 5048–5058, 2017.
- [3] J Andrew Bagnell. An invitation to imitation. Technical report, CARNEGIE-MELLON UNIV PITTSBURGH PA ROBOTICS INST, 2015.
- [4] J Andrew Bagnell and Jeff Schneider. Covariant policy search. 2003.
- [5] Amir Beck. *First-order methods in optimization*, volume 25. SIAM-Society for Industrial and Applied Mathematics, 2017.
- [6] Dimitri P Bertsekas and John N Tsitsiklis. *Neuro-dynamic programming*. Athena Scientific, 1996.
- [7] Jeannette Bohg, Antonio Morales, Tamim Asfour, and Danica Kragic. Data-driven grasp synthesis survey. *IEEE Transactions on Robotics*, 30(2):289–309, 2013.
- [8] Sofien Bouaziz, Andrea Tagliasacchi, and Mark Pauly. Sparse iterative closest point. In *Computer graphics forum*, volume 32, pages 113–123. Wiley Online Library, 2013.
- [9] Douglas A Bristow, Marina Tharayil, and Andrew G Alleyne. A survey of iterative learning control. *IEEE control systems magazine*, 26(3):96–114, 2006.
- [10] Sébastien Bubeck and Nicolo Cesa-Bianchi. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends in Machine Learning*, 5(1):1–122, 2012.
- [11] Berk Calli, Aaron Walsman, Arjun Singh, Siddhartha Srinivasa, Pieter Abbeel, and Aaron Dollar. Benchmarking in manipulation research: The YCB object and model set and benchmarking protocols. *IEEE Robotics and Automation Magazine*, 22(3):36–52, 2015.
- [12] I-Ming Chen and Joel Burdick. Finding antipodal point grasps on irregularly shaped objects. *IEEE transactions on Robotics and Automation*, 9(4):507–512, 1993.

- [13] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*, 2015.
- [14] Ricson Cheng, Arpit Agarwal, and Katerina Fragkiadaki. Reinforcement learning of active vision for manipulating objects under occlusions. *arXiv preprint arXiv:1811.08067*, 2018.
- [15] Dmitry Chetverikov, Dmitry Svirko, Dmitry Stepanov, and Pavel Krsek. The trimmed iterative closest point algorithm. In *Object recognition supported by user interaction for service robots*, volume 3, pages 545–548. IEEE, 2002.
- [16] Matei Ciocarlie, Corey Goldfeder, and Peter Allen. Dexterous grasping via eigengrasps: A low-dimensional approach to a high-complexity problem.
- [17] Matei Ciocarlie, Corey Goldfeder, and Peter Allen. Dexterous grasping via eigengrasps: A low-dimensional approach to a high-complexity problem. In *Robotics: Science and Systems Manipulation Workshop-Sensing and Adapting to the Real World (RSS)*, 2007.
- [18] Ignasi Clavera, Violet Fu, and Pieter Abbeel. Model-augmented actor-critic: Backpropagating through paths. *arXiv preprint arXiv:2005.08068*, 2020.
- [19] Nikolaus Correll, Kostas E Bekris, Dmitry Berenson, Oliver Brock, Albert Causo, Kris Hauser, Kei Okada, Alberto Rodriguez, Joseph M Romano, and Peter R Wurman. Analysis and observations from the first amazon picking challenge. *IEEE Transactions on Automation Science and Engineering*, 15(1):172–188, 2016.
- [20] Erwin Coumans and Yunfei Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>, 2016–2019.
- [21] Xinke Deng, Arsalan Mousavian, Yu Xiang, Fei Xia, Timothy Bretl, and Dieter Fox. Poserbpf: A rao-blackwellized particle filter for 6d object pose tracking. *arXiv preprint arXiv:1905.09304*, 2019.
- [22] Xinke Deng, Yu Xiang, Arsalan Mousavian, Clemens Eppner, Timothy Bretl, and Dieter Fox. Self-supervised 6d object pose estimation for robot manipulation. *arXiv preprint arXiv:1909.10159*, 2019.
- [23] Yiming Ding, Carlos Florensa, Pieter Abbeel, and Mariano Phielipp. Goal-conditioned imitation learning. In *Advances in Neural Information Processing Systems*, pages 15298–15309, 2019.
- [24] Alexey Dosovitskiy, Philipp Fischer, Eddy Ilg, Philip Hausser, Caner Hazirbas, Vladimir Golkov, Patrick Van Der Smagt, Daniel Cremers, and Thomas Brox. FlowNet: Learning optical flow with convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2758–2766, 2015.

- [25] Anca Dragan, Geoffrey Gordon, and Siddhartha Srinivasa. Learning from experience in manipulation planning: Setting the right goals. In *International Symposium on Robotics Research (ISRR)*, 2011.
- [26] Anca Dragan, Nathan Ratliff, and Siddhartha Srinivasa. Manipulation planning with goal sets using constrained trajectory optimization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4582–4588, 2011.
- [27] Frederik Ebert, Chelsea Finn, Sudeep Dasari, Annie Xie, Alex Lee, and Sergey Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *arXiv preprint arXiv:1812.00568*, 2018.
- [28] Clemens Eppner, Arsalan Mousavian, and Dieter Fox. A billion ways to grasp: An evaluation of grasp sampling schemes on a dense, physics-based grasp data set. In *International Symposium on Robotics Research (ISRR)*, 2019.
- [29] Yongxiang Fan, Hsien-Chung Lin, Te Tang, and Masayoshi Tomizuka. Grasp planning for customized grippers by iterative surface fitting. In *IEEE International Conference on Automation Science and Engineering (CASE)*, pages 28–34, 2018.
- [30] Kuan Fang, Yuke Zhu, Animesh Garg, Silvio Savarese, and Li Fei-Fei. Dynamics learning with cascaded variational inference for multi-step manipulation. *arXiv preprint arXiv:1910.13395*, 2019.
- [31] C. Ferrari and J. Canny. Planning optimal grasps. In *Proceedings 1992 IEEE International Conference on Robotics and Automation*, pages 2290–2295 vol.3, May 1992.
- [32] Carlo Ferrari and John Canny. Planning optimal grasps. In *IEEE International Conference on Robotics and Automation (ICRA)*, volume 3, pages 2290–2295, 1992.
- [33] Chelsea Finn, Paul Christiano, Pieter Abbeel, and Sergey Levine. A connection between generative adversarial networks, inverse reinforcement learning, and energy-based models. *arXiv preprint arXiv:1611.03852*, 2016.
- [34] Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *International conference on machine learning*, pages 49–58, 2016.
- [35] Joan Fontanals, Bao-Anh Dang-Vu, Oliver Porges, Jan Rosell, and Máximo Roa. Integrated grasp and motion planning using independent contact regions. In *IEEE-RAS International Conference on Humanoid Robots*, pages 887–893, 2014.
- [36] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Sampling-based methods for factored task and motion planning. *The International Journal of Robotics Research (IJRR)*, 37(13-14):1796–1825, 2018.

- [37] Dibya Ghosh, Abhishek Gupta, Justin Fu, Ashwin Reddy, Coline Devine, Benjamin Eysenbach, and Sergey Levine. Learning to reach goals without reinforcement learning. *arXiv preprint arXiv:1912.06088*, 2019.
- [38] Julio Godoy, Ioannis Karamouzas, Stephen Guy, and Maria Gini. Adaptive learning for multi-agent navigation. In *The International Conference on Autonomous Agents and Multiagent Systems*, pages 1577–1585, 2015.
- [39] Corey Goldfeder and Peter Allen. Data-driven grasping. *Autonomous Robots*, 31(1):1–20, 2011.
- [40] Corey Goldfeder, Matei Ciocarlie, Hao Dang, and Peter K Allen. The columbia grasp database. In *2009 IEEE international conference on robotics and automation*, pages 1710–1716. IEEE, 2009.
- [41] Lars Grüne and Jürgen Pannek. Nonlinear model predictive control. In *Nonlinear Model Predictive Control*, pages 45–69. Springer, 2017.
- [42] Marcus Gualtieri and Robert Platt. Viewpoint selection for grasp detection. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 258–264. IEEE, 2017.
- [43] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.
- [44] Joshua Haustein, Kaiyu Hang, and Danica Kragic. Integrating motion and hierarchical fingertip grasp planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3439–3446, 2017.
- [45] Elad Hazan. Introduction to online convex optimization. *Foundations and Trends in Optimization*, 2(3-4):157–325, 2016.
- [46] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.
- [47] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [48] Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Stefan Holzer, Gary Bradski, Kurt Konolige, and Nassir Navab. Model based training, detection and pose estimation of textureless 3d objects in heavily cluttered scenes. In *Asian conference on computer vision*, pages 548–562. Springer, 2012.

- [49] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in neural information processing systems*, pages 4565–4573, 2016.
- [50] Lucas Janson, Edward Schmerling, Ashley Clark, and Marco Pavone. Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions. *The International Journal of Robotics Research (IJRR)*, 34(7):883–921, 2015.
- [51] Leslie Pack Kaelbling. Learning to achieve goals. In *IJCAI*, pages 1094–1099. Citeseer, 1993.
- [52] Sham M Kakade. A natural policy gradient. In *Advances in neural information processing systems*, pages 1531–1538, 2002.
- [53] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. STOMP: Stochastic trajectory optimization for motion planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4569–4574, 2011.
- [54] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293*, 2018.
- [55] Sertac Karaman and Emilio Frazzoli. Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research (IJRR)*, 30(7):846–894, 2011.
- [56] Lydia Kavraki and Jean-Claude Latombe. Practical motion planning in robotics: Current approaches and future directions, chapter probabilistic roadmaps for robot path planning, 1998.
- [57] Wadim Kehl, Fabian Manhardt, Federico Tombari, Slobodan Ilic, and Nassir Navab. Ssd-6d: Making rgb-based 3d detection and 6d pose estimation great again. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1521–1529, 2017.
- [58] Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, pages 1008–1014, 2000.
- [59] Oliver Kroemer, Renaud Detry, Justus Piater, and Jan Peters. Combining active learning and reactive control for robot grasping. *Robotics and Autonomous systems*, 58(9):1105–1116, 2010.
- [60] James Kuffner and Steven LaValle. RRT-connect: An efficient approach to single-query path planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, volume 2, pages 995–1001, 2000.

- [61] Thanard Kurutach, Ignasi Clavera, Yan Duan, Aviv Tamar, and Pieter Abbeel. Model-ensemble trust-region policy optimization. *arXiv preprint arXiv:1802.10592*, 2018.
- [62] Fabien Lagriffoul, Dimitar Dimitrov, Julien Bidot, Alessandro Saffiotti, and Lars Karlsson. Efficiently combining task and motion planning using geometric constraints. *The International Journal of Robotics Research (IJRR)*, 33(14):1726–1747, 2014.
- [63] Tor Lattimore and Csaba Szepesvári. Bandit algorithms. *preprint*, 2018.
- [64] Steven LaValle. Rapidly-exploring random trees: A new tool for path planning. *Technical Report. Computer Science Department, Iowa State University*, 1998.
- [65] Vincent Lepetit, Francesc Moreno-Noguer, and Pascal Fua. Eppn: An accurate $O(n)$ solution to the pnp problem. *International journal of computer vision*, 81(2):155, 2009.
- [66] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.
- [67] Sergey Levine and Vladlen Koltun. Guided policy search. In *International Conference on Machine Learning*, pages 1–9, 2013.
- [68] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, 37(4-5):421–436, 2018.
- [69] Ke Li, Shichong Peng, Tianhao Zhang, and Jitendra Malik. Multimodal image synthesis with conditional implicit maximum likelihood estimation. *arXiv preprint arXiv:2004.03590*, 2020.
- [70] Yi Li, Gu Wang, Xiangyang Ji, Yu Xiang, and Dieter Fox. Deepim: Deep iterative matching for 6d pose estimation. In *European Conference on Computer Vision (ECCV)*, 2018.
- [71] Kendall Lowrey, Aravind Rajeswaran, Sham Kakade, Emanuel Todorov, and Igor Mordatch. Plan online, learn offline: Efficient learning and exploration via model-based control. *arXiv preprint arXiv:1811.01848*, 2018.
- [72] Jens Lundell. 6-dof graspnet pytorch. *<https://github.com/jsll/pytorch6dof-graspnet>*.
- [73] Ajay Mandlekar, Fabio Ramos, Byron Boots, Li Fei-Fei, Animesh Garg, and Dieter Fox. Iris: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data. *arXiv preprint arXiv:1911.05321*, 2019.
- [74] Zita Marinho, Anca Dragan, Arun Byravan, Byron Boots, Siddhartha Srinivasa, and Geoffrey Gordon. Functional gradient motion planning in reproducing kernel hilbert spaces. *arXiv preprint arXiv:1601.03648*, 2016.

- [75] Andrew Miller and Peter Allen. Graspit! A versatile simulator for robotic grasping. *IEEE Robotics & Automation Magazine*, 11(4):110–122, 2004.
- [76] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [77] William H Montgomery and Sergey Levine. Guided policy search via approximate mirror descent. In *Advances in Neural Information Processing Systems*, pages 4008–4016, 2016.
- [78] Igor Mordatch, Zoran Popović, and Emanuel Todorov. Contact-invariant optimization for hand manipulation. In *Proceedings of the ACM SIGGRAPH/Eurographics symposium on computer animation*, pages 137–144. Eurographics Association, 2012.
- [79] Douglas Morrison, Peter Corke, and Jürgen Leitner. Closing the Loop for Robotic Grasping: A Real-time, Generative Grasp Synthesis Approach. In *Proc. of Robotics: Science and Systems (RSS)*, 2018.
- [80] Arsalan Mousavian, Clemens Eppner, and Dieter Fox. 6-DOF GraspNet: Variational grasp generation for object manipulation. In *International Conference on Computer Vision (ICCV)*, pages 2901–2910, 2019.
- [81] Mustafa Mukadam, Jing Dong, Xinyan Yan, Frank Dellaert, and Byron Boots. Continuous-time gaussian process motion planning via probabilistic inference. *The International Journal of Robotics Research (IJRR)*, 37(11):1319–1340, 2018.
- [82] Ashvin Nair, Bob McGrew, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6292–6299. IEEE, 2018.
- [83] Ashvin V Nair, Vitchyr Pong, Murtaza Dalal, Shikhar Bahl, Steven Lin, and Sergey Levine. Visual reinforcement learning with imagined goals. In *Advances in Neural Information Processing Systems*, pages 9191–9200, 2018.
- [84] Andrew Y Ng, Stuart J Russell, et al. Algorithms for inverse reinforcement learning. In *ICML*, volume 1, pages 663–670, 2000.
- [85] Van-Duc Nguyen. Constructing force-closure grasps. *The International Journal of Robotics Research (IJRR)*, 7(3):3–16, 1988.
- [86] Takayuki Osa. Multimodal trajectory optimization for motion planning. *arXiv preprint arXiv:2003.07054*, 2020.

- [87] Takayuki Osa, Joni Pajarinen, Gerhard Neumann, J Andrew Bagnell, Pieter Abbeel, and Jan Peters. An algorithmic perspective on imitation learning. *arXiv preprint arXiv:1811.06711*, 2018.
- [88] Lerrel Pinto and Abhinav Gupta. Supersizing self-supervision: Learning to grasp from 50k tries and 700 robot hours. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 3406–3413. IEEE, 2016.
- [89] D Pomerleau. An autonomous land vehicle in a neural network. *Advances in neural information processing systems(Morgan Kaufmann Publishers Inc., 1989)*, 1, 1998.
- [90] Iyaylo Popov, Nicolas Heess, Timothy Lillicrap, Roland Hafner, Gabriel Barth-Maron, Matej Vecerik, Thomas Lampe, Yuval Tassa, Tom Erez, and Martin Riedmiller. Data-efficient deep reinforcement learning for dexterous manipulation. *arXiv preprint arXiv:1704.03073*, 2017.
- [91] Michael Posa, Cecilia Cantu, and Russ Tedrake. A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics Research*, 33(1):69–81, 2014.
- [92] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in neural information processing systems*, pages 5099–5108, 2017.
- [93] Deirdre Quillen, Eric Jang, Ofir Nachum, Chelsea Finn, Julian Ibarz, and Sergey Levine. Deep reinforcement learning for vision-based robotic grasping: A simulated comparative evaluation of off-policy methods. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6284–6291. IEEE, 2018.
- [94] Mahdi Rad and Vincent Lepetit. Bb8: A scalable, accurate, robust to partial occlusion method for predicting the 3d poses of challenging objects without using depth. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 3828–3836, 2017.
- [95] Aravind Rajeswaran, Vikash Kumar, Abhishek Gupta, Giulia Vezzani, John Schulman, Emanuel Todorov, and Sergey Levine. Learning complex dexterous manipulation with deep reinforcement learning and demonstrations. *arXiv preprint arXiv:1709.10087*, 2017.
- [96] Aravind Rajeswaran, Kendall Lowrey, Emanuel V Todorov, and Sham M Kakade. Towards generalization and simplicity in continuous control. In *Advances in Neural Information Processing Systems*, pages 6550–6561, 2017.
- [97] Aravind Rajeswaran, Igor Mordatch, and Vikash Kumar. A game theoretic framework for model based reinforcement learning. *arXiv preprint arXiv:2004.07804*, 2020.

- [98] Nathan Ratliff, Matt Zucker, Andrew Bagnell, and Siddhartha Srinivasa. CHOMP: Gradient optimization techniques for efficient motion planning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [99] Joseph Redmon and Anelia Angelova. Real-time grasp detection using convolutional neural networks. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1316–1322, 2015.
- [100] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [101] Ugo Rosolia and Francesco Borrelli. Learning model predictive control for iterative tasks. a data-driven control framework. *IEEE Transactions on Automatic Control*, 63(7):1883–1896, 2017.
- [102] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635, 2011.
- [103] Fred Rothganger, Svetlana Lazebnik, Cordelia Schmid, and Jean Ponce. 3d object modeling and recognition using local affine-invariant image descriptors and multi-view spatial constraints. *International journal of computer vision*, 66(3):231–259, 2006.
- [104] Adam Santoro, David Raposo, David G Barrett, Mateusz Malinowski, Razvan Pascanu, Peter Battaglia, and Timothy Lillicrap. A simple neural network module for relational reasoning. In *Advances in neural information processing systems*, pages 4967–4976, 2017.
- [105] Tanner Schmidt, Richard A Newcombe, and Dieter Fox. Dart: Dense articulated real-time tracking. In *Robotics: Science and Systems*, volume 2. Berkeley, CA, 2014.
- [106] John Schulman, Yan Duan, Jonathan Ho, Alex Lee, Ibrahim Awwal, Henry Bradlow, Jia Pan, Sachin Patil, Ken Goldberg, and Pieter Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research (IJRR)*, 33(9):1251–1270, 2014.
- [107] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897, 2015.
- [108] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [109] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. 2014.

- [110] Avi Singh, Larry Yang, Kristian Hartikainen, Chelsea Finn, and Sergey Levine. End-to-end robotic reinforcement learning without reward engineering. *arXiv preprint arXiv:1904.07854*, 2019.
- [111] Shuran Song, Andy Zeng, Johnny Lee, and Thomas Funkhouser. Grasping in the wild: Learning 6dof closed-loop grasping from low-cost demonstrations. *arXiv preprint arXiv:1912.04344*, 2019.
- [112] Siddharth Srivastava, Eugene Fang, Lorenzo Riano, Rohan Chitnis, Stuart Russell, and Pieter Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 639–646, 2014.
- [113] Wen Sun, J Andrew Bagnell, and Byron Boots. Truncated horizon policy search: Combining reinforcement learning & imitation learning. *arXiv preprint arXiv:1805.11240*, 2018.
- [114] Wen Sun, Arun Venkatraman, Geoffrey J Gordon, Byron Boots, and J Andrew Bagnell. Deeply aggravated: Differentiable imitation learning for sequential prediction. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pages 3309–3318. JMLR. org, 2017.
- [115] Martin Sundermeyer, Zoltan-Csaba Marton, Maximilian Durner, Manuel Brucker, and Rudolph Triebel. Implicit 3d orientation learning for 6d object detection from rgb images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 699–715, 2018.
- [116] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
- [117] Andreas ten Pas, Marcus Gualtieri, Kate Saenko, and Robert Platt. Grasp pose detection in point clouds. *Int. J. Rob. Res.*, 36(13-14):1455–1473, December 2017.
- [118] Andreas ten Pas, Marcus Gualtieri, Kate Saenko, and Robert Platt. Grasp pose detection in point clouds. *The International Journal of Robotics Research (IJRR)*, 36(13-14):1455–1473, 2017.
- [119] Evangelos Theodorou, Jonas Buchli, and Stefan Schaal. A generalized path integral control approach to reinforcement learning. *journal of machine learning research*, 11(Nov):3137–3181, 2010.
- [120] Emanuel Todorov and Weiwei Li. A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems. In *Proceedings of the 2005, American Control Conference, 2005.*, pages 300–306. IEEE, 2005.

- [121] Marc Toussaint. Logic-geometric programming: An optimization-based approach to combined task and motion planning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
- [122] Marc Toussaint, Kelsey Allen, Kevin Smith, and Joshua Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. In *Robotics: Science and Systems (RSS)*, 2018.
- [123] John N Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. In *Advances in neural information processing systems*, pages 1075–1081, 1997.
- [124] Nikolaus Vahrenkamp, Martin Do, Tamim Asfour, and Rüdiger Dillmann. Integrated grasp and motion planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2883–2888, 2010.
- [125] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Thirtieth AAAI conference on artificial intelligence*, 2016.
- [126] Mel Vecerik, Todd Hester, Jonathan Scholz, Fumin Wang, Olivier Pietquin, Bilal Piot, Nicolas Heess, Thomas Rothörl, Thomas Lampe, and Martin Riedmiller. Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards. *arXiv preprint arXiv:1707.08817*, 2017.
- [127] Nolan Wagener, Ching-An Cheng, Jacob Sacks, and Byron Boots. An online learning approach to model predictive control. In *Robotics: Science and Systems (RSS)*, 2019.
- [128] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. Densefusion: 6d object pose estimation by iterative dense fusion. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3343–3352, 2019.
- [129] Lirui Wang, Yu Xiang, and Dieter Fox. Manipulation trajectory optimization with online grasp synthesis and selection. *arXiv preprint arXiv:1911.10280*, 2019.
- [130] Tingwu Wang, Xuchan Bao, Ignasi Clavera, Jerrick Hoang, Yeming Wen, Eric Langlois, Shunshi Zhang, Guodong Zhang, Pieter Abbeel, and Jimmy Ba. Benchmarking model-based reinforcement learning. *arXiv preprint arXiv:1907.02057*, 2019.
- [131] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.
- [132] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. 2018.

- [133] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. *arXiv preprint arXiv:1910.10897*, 2019.
- [134] Yuke Zhu, Ziyu Wang, Josh Merel, Andrei Rusu, Tom Erez, Serkan Cabi, Saran Tunyasuvunakool, János Kramár, Raia Hadsell, Nando de Freitas, et al. Reinforcement and imitation learning for diverse visuomotor skills. *arXiv preprint arXiv:1802.09564*, 2018.
- [135] Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, and Anind K Dey. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008.